

LeetCode "Blind 75" Deep Dive Study Guide

Curated by Senior FAANG Interviewers & MLSys Engineers

Table of Contents

Arrays

- 01. [LeetCode 1: Two Sum](#)
- 02. [LeetCode 121: Best Time to Buy and Sell Stock](#)
- 03. [LeetCode 217: Contains Duplicate](#)
- 04. [LeetCode 238: Product of Array Except Self](#)
- 05. [LeetCode 53: Maximum Subarray](#)
- 06. [LeetCode 152: Maximum Product Subarray](#)
- 07. [LeetCode 153: Find Minimum in Rotated Sorted Array](#)
- 08. [LeetCode 33: Search in Rotated Sorted Array](#)
- 09. [LeetCode 15: 3Sum](#)
- 10. [LeetCode 11: Container With Most Water](#)

Binary

- 11. [LeetCode 371: Sum of Two Integers](#)
- 12. [LeetCode 191: Number of 1 Bits](#)
- 13. [LeetCode 338: Counting Bits](#)
- 14. [LeetCode 268: Missing Number](#)

15. [LeetCode 190: Reverse Bits](#)

Dynamic Programming

16. [LeetCode 70: Climbing Stairs](#)

17. [LeetCode 322: Coin Change](#)

18. [LeetCode 300: Longest Increasing Subsequence](#)

19. [LeetCode 1143: Longest Common Subsequence](#)

20. [LeetCode 139: Word Break](#)

21. [LeetCode 377: Combination Sum IV](#)

22. [LeetCode 198: House Robber](#)

23. [LeetCode 213: House Robber II](#)

24. [LeetCode 91: Decode Ways](#)

25. [LeetCode 62: Unique Paths](#)

26. [LeetCode 55: Jump Game](#)

Graphs

27. [LeetCode 133: Clone Graph](#)

28. [LeetCode 207: Course Schedule](#)

29. [LeetCode 417: Pacific Atlantic Water Flow](#)

30. [LeetCode 200: Number of Islands](#)

31. [LeetCode 128: Longest Consecutive Sequence](#)

32. [LeetCode 269: Alien Dictionary](#)

33. [LeetCode 261: Graph Valid Tree](#)

34. [LeetCode 323: Number of Connected Components in an Undirected Graph](#)

Intervals

35. [LeetCode 57: Insert Interval](#)

36. [LeetCode 56: Merge Intervals](#)

37. [LeetCode 435: Non-overlapping Intervals](#)

38. [LeetCode 252: Meeting Rooms](#)

39. [LeetCode 253: Meeting Rooms II](#)

Linked Lists

- 40. LeetCode 206: Reverse Linked List
- 41. LeetCode 141: Linked List Cycle
- 42. LeetCode 21: Merge Two Sorted Lists
- 43. LeetCode 23: Merge k Sorted Lists
- 44. LeetCode 19: Remove Nth Node From End of List
- 45. LeetCode 143: Reorder List

Matrices

- 46. LeetCode 73: Set Matrix Zeroes
- 47. LeetCode 54: Spiral Matrix
- 48. LeetCode 48: Rotate Image
- 49. LeetCode 79: Word Search

Strings

- 50. LeetCode 3: Longest Substring Without Repeating Characters
- 51. LeetCode 424: Longest Repeating Character Replacement
- 52. LeetCode 76: Minimum Window Substring
- 53. LeetCode 242: Valid Anagram
- 54. LeetCode 49: Group Anagrams
- 55. LeetCode 20: Valid Parentheses
- 56. LeetCode 125: Valid Palindrome
- 57. LeetCode 5: Longest Palindromic Substring
- 58. LeetCode 647: Palindromic Substrings
- 59. LeetCode 271: Encode and Decode Strings

Trees

- 60. LeetCode 104: Maximum Depth of Binary Tree
- 61. LeetCode 100: Same Tree
- 62. LeetCode 226: Invert Binary Tree
- 63. LeetCode 124: Binary Tree Maximum Path Sum
- 64. LeetCode 102: Binary Tree Level Order Traversal
- 65. LeetCode 297: Serialize and Deserialize Binary Tree

- 66. LeetCode 572: Subtree of Another Tree
- 67. LeetCode 105: Construct Binary Tree from Preorder and Inorder Traversal
- 68. LeetCode 98: Validate Binary Search Tree
- 69. LeetCode 230: Kth Smallest Element in a BST
- 70. LeetCode 235: Lowest Common Ancestor of a Binary Search Tree
- 71. LeetCode 208: Implement Trie (Prefix Tree)
- 72. LeetCode 211: Add and Search Word
- 73. LeetCode 212: Word Search II

Heaps

- 74. LeetCode 347: Top K Frequent Elements
- 75. LeetCode 295: Find Median from Data Stream

Arrays

01. [LeetCode 1: Two Sum](#) [Easy]

 **Key Points:** Hash Map - Space-Time Tradeoff

Intuition & Breaking Points:

Intuition & Pitfalls: The naive approach is using nested loops to check every pair of elements for a sum equal to the target, which costs $O(N^2)$ time complexity. This is inefficient for large inputs. We need to avoid using the same element twice and handle duplicate values correctly.

Mathematical Derivation: To optimize the time complexity to $O(N)$, we can trade space for time. By traversing the array once and storing each visited element and its index in a hash map, we can check if the complement ($\text{target} - \text{current_value}$) exists in the hash map in $O(1)$ average time. If it does, we return their indices.

```
from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        # Hash map to store value-to-index mapping
        num_to_idx = {}
        for i, num in enumerate(nums):
            complement = target - num
            # Check if the complement already exists in the map
            if complement in num_to_idx:
                return [num_to_idx[complement], i]
            # Store the current number with its index
```

```
num_to_idx[num] = i
return []
```

02. [LeetCode 121: Best Time to Buy and Sell Stock](#) [Easy]

 **Key Points:** Greedy / Dynamic Programming - Single Pass

Intuition & Breaking Points:

Intuition & Pitfalls: The brute-force solution calculates the profit for every possible buy-and-sell pair (where sell day > buy day) using nested loops, which takes $O(N^2)$ time complexity.

Mathematical Derivation: Since you must buy before you can sell, we can solve this in a single pass. While traversing the prices, we maintain two variables: the minimum price seen so far (`min\_price`) and the maximum profit achieved (`max\_profit`). For each price, we update `min\_price` and calculate the potential profit if we sold on that day (`price - min\_price`), updating `max\_profit` if this potential profit is larger.

```
from typing import List

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        if not prices:
            return 0

        min_price = float('inf') # Track the minimum price seen so far
        max_profit = 0           # Track the maximum profit seen so far
        for price in prices:
            if price < min_price:
                min_price = price # Update min price if a lower buying price is found
            elif price - min_price > max_profit:
                max_profit = price - min_price # Update max profit if selling today yields
```

```
more
```

```
return max_profit
```

03. [LeetCode 217: Contains Duplicate](#) [Easy]

 **Key Points:** Hash Set - Early Return Optimization

Intuition & Breaking Points:

Intuition & Pitfalls: A brute-force solution compares every pair of elements, resulting in $O(N^2)$ time complexity. Using `len(set(nums)) != len(nums)` is clean but does not allow early return, which might waste memory and computation on large arrays when a duplicate is found early.

Mathematical Derivation: To achieve $O(N)$ time complexity, we can use a hash set to keep track of visited numbers. As we iterate through the array, we check if the current number is already in the set. If it is, we return True immediately (early return). Otherwise, we add it to the set. If the loop completes, it means all elements are unique.

```
from typing import List

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        visited = set() # Set to track unique visited numbers
        for num in nums:
            # If the number is already visited, we found a duplicate
            if num in visited:
                return True
            visited.add(num) # Add current number to visited set
        return False
```

04. [LeetCode 238: Product of Array Except Self](#) [Medium]

 **Key Points:** Prefix & Suffix Products - Space Optimization

Intuition & Breaking Points:

Intuition & Pitfalls: The simplest method is to calculate the product of all elements and then divide it by each `nums[i]`. However, division is prohibited, and division by zero errors will occur if the array contains zero.

Mathematical Derivation: The product of all elements except `nums[i]` can be decomposed into: the product of all elements to the left of `i` (prefix product) multiplied by the product of all elements to the right of `i` (suffix product). We can store the prefix products directly in the output array. Then, scanning backwards, we maintain a running suffix product in a variable and multiply it into the output array at each index. This achieves $O(1)$ auxiliary space.

```
from typing import List

class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        answer = [1] * n

        # Calculate prefix products and store them in the answer array
        prefix_product = 1
        for i in range(n):
            answer[i] = prefix_product
            prefix_product *= nums[i]

        # Calculate suffix products on the fly and multiply them into answer
        suffix_product = 1
        for i in range(n - 1, -1, -1):
            answer[i] *= suffix_product
            suffix_product *= nums[i]
```

```
return answer
```

05. [LeetCode 53: Maximum Subarray](#) [Medium]

🔑 **Key Points:** Dynamic Programming (Kadane's Algorithm)

💡 Intuition & Breaking Points:

Intuition & Pitfalls: Brute-force checks all subarray starting and ending positions, which takes $O(N^2)$ time complexity. If the array contains only negative numbers, initializing the maximum subarray sum to 0 is a common mistake; it must be initialized to the first element or negative infinity.

Mathematical Derivation: At each position in the array, we decide whether to add the current number to the existing subarray sum, or to start a new subarray from the current number. If the previous cumulative sum is negative, it will only decrease the sum of any subsequent subarray, so we discard it and start fresh. The state transition is: `current_sum = max(num, current_sum + num)`. We track the maximum value seen during this process.

```
from typing import List

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        current_sum = nums[0] # Max subarray sum ending at current index
        max_sum = nums[0]     # Global max subarray sum

        for num in nums[1:]:
            # Choose to extend the previous subarray or start a new one
            current_sum = max(num, current_sum + num)
            max_sum = max(max_sum, current_sum)
```

```
return max_sum
```

06. [LeetCode 152: Maximum Product Subarray](#) [Medium]

 **Key Points:** Dynamic Programming - Double State Tracking

Intuition & Breaking Points:

Intuition & Pitfalls: Unlike the subarray sum problem, multiplication supports 'two negatives make a positive'. A very small negative number can become a large positive number when multiplied by another negative. Thus, tracking only the maximum product is insufficient.

Mathematical Derivation: We must maintain both the current maximum product and the current minimum product (which could be a large negative number). For each element, we calculate the potential new max and min by comparing the current number, current number * prev_max, and current number * prev_min. This allows us to handle sign flips in $O(N)$ time and $O(1)$ space.

```
from typing import List

class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        if not nums:
            return 0
        res = nums[0]
        cur_min = nums[0] # Minimum product seen ending at current index
        cur_max = nums[0] # Maximum product seen ending at current index

        for num in nums[1:]:
            temp = cur_max
            # Update current max and min considering possible sign changes
            cur_max = max(num, num * cur_max, num * cur_min)
            cur_min = min(num, num * temp, num * cur_min)
```

```
res = max(res, cur_max)

return res
```

07. [LeetCode 153: Find Minimum in Rotated Sorted Array](#) [Medium]

 **Key Points:** Binary Search

Intuition & Breaking Points:

Intuition & Pitfalls: Finding the minimum in a rotated sorted array. A linear scan takes $O(N)$. Because the array was originally sorted, we must solve it in $O(\log N)$ time using binary search.

Mathematical Derivation: Using binary search, if the middle value `nums[mid]` is greater than the right boundary `nums[right]`, it means the rotation inflection point lies to the right of `mid`, so the minimum must be in the range `[mid + 1, right]`. Otherwise, the right half is sorted, and the minimum is either `nums[mid]` or lies to the left of `mid`. Thus, we update `right = mid`. We repeat this until `left == right`.

```
from typing import List

class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            # Inflection point must be to the right of mid
            if nums[mid] > nums[right]:
                left = mid + 1
            # Inflection point is at mid or to the left of mid
            else:
```

```

        right = mid
    return nums[left]

```

08. [LeetCode 33: Search in Rotated Sorted Array](#) [Medium]

 **Key Points:** Binary Search - Split Monotonicity

Intuition & Breaking Points:

Intuition & Pitfalls: Searching for a target in a rotated sorted array. A linear scan takes $O(N)$. We need to leverage binary search to achieve $O(\log N)$ complexity.

Mathematical Derivation: A rotated sorted array has a key property: if split in half, at least one half is always strictly sorted. We can compare `nums[left]` and `nums[mid]` to determine which half is sorted: 1. If the left half is sorted, we check if the target lies within the range of this sorted half. If so, we search left (`right = mid - 1`), else we search right (`left = mid + 1`). 2. If the right half is sorted, we check if the target lies within the range of this sorted half, adjusting the binary search bounds accordingly.

```

from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            # Check if left half is sorted
            if nums[left] <= nums[mid]:
                # Check if target is inside the sorted left half
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
            else:

```



```

        left = mid + 1
    # Otherwise, right half must be sorted
    else:
        # Check if target is inside the sorted right half
        if nums[mid] < target <= nums[right]:
            left = mid + 1
        else:
            right = mid - 1
    return -1

```

09. [LeetCode 15: 3Sum](#) [Medium]

 **Key Points:** Two Pointers - Sorting

Intuition & Breaking Points:

Intuition & Pitfalls: A brute-force search of three numbers takes $O(N^3)$ time complexity and has extremely tedious duplicate removal logic. We need an efficient way to sort, prune, and skip duplicate elements.

Mathematical Derivation: 1. First, sort the array in ascending order to use two pointers. 2. Fix the first number `nums[i]`. If `nums[i] > 0`, it's impossible to sum to 0 with subsequent positive numbers, so we break. If `nums[i] == nums[i-1]`, skip it to avoid duplicate triplets. 3. For the remaining range `[i + 1, n - 1]`, use two pointers `left` and `right`. If the sum is less than 0, increment `left`; if greater than 0, decrement `right`. If equal to 0, record the triplet, and shift both pointers while skipping duplicate values to prevent duplicates.

```

from typing import List

class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort() # Sort first to enable two pointers
        res = []

```

```

n = len(nums)
for i in range(n - 2):
    if nums[i] > 0:
        break # Pivot cannot be positive
    if i > 0 and nums[i] == nums[i - 1]:
        continue # Skip duplicate pivots

    left, right = i + 1, n - 1
    while left < right:
        total = nums[i] + nums[left] + nums[right]
        if total < 0:
            left += 1
        elif total > 0:
            right -= 1
        else:
            res.append([nums[i], nums[left], nums[right]])
            # Shift left and right pointers while skipping duplicates
            while left < right and nums[left] == nums[left + 1]:
                left += 1
            while left < right and nums[right] == nums[right - 1]:
                right -= 1
            left += 1
            right -= 1

    return res

```

10. [LeetCode 11: Container With Most Water](#) [Medium]

 **Key Points:** Two Pointers - Greedy Pointer Shifting

Intuition & Breaking Points:

Intuition & Pitfalls: Calculating the volume for every possible pair of lines takes $O(N^2)$ time complexity. We can leverage the boundaries and solve this in $O(N)$ using two pointers.

Mathematical Derivation: We place two pointers at the two ends of the array. The width of the container is `right - left`, and the height is bounded by the shorter line

`min(height[left], height[right])`. Each time, we shift the pointer pointing to the shorter line inward. Shifting the pointer pointing to the longer line cannot increase the area because the height is still capped by the shorter line, while the width decreases. Thus, moving the shorter line is the only greedy strategy that could potentially yield a larger area.

```
from typing import List

class Solution:
    def maxArea(self, height: List[int]) -> int:
        left, right = 0, len(height) - 1
        max_water = 0
        while left < right:
            width = right - left
            current_height = min(height[left], height[right])
            max_water = max(max_water, width * current_height)
            # Greedily move the pointer pointing to the shorter line
            if height[left] < height[right]:
                left += 1
            else:
                right -= 1
        return max_water
```

Binary

11. [LeetCode 371: Sum of Two Integers](#) [Medium]

 **Key Points:** Bit Manipulation - XOR for carry-less addition, AND and Shift for carry

Intuition & Breaking Points:

Intuition & Pitfalls: To add two integers without using '+' or '-', we must simulate adder circuits at the binary level. Addition of two numbers can be split into: carry-less sum (which can be calculated using XOR) and the carry bits (which can be calculated using AND followed by a left shift of 1).

Mathematical Derivation: For two integers a and b: 1. $a \oplus b$ calculates the carry-less sum (e.g., $1+1=0$, $1+0=1$). 2. $(a \& b) \ll 1$ calculates the carry bits (since only $1+1$ creates a carry, which is shifted left by 1). 3. Repeat this process recursively/iteratively until the carry b becomes 0. In Python, integers have arbitrary precision, so we must manually apply a 32-bit mask `0xFFFFFFFF` to simulate the overflow and negative bounds of a standard 32-bit signed integer.

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        # Time Complexity: O(1) - Loop runs at most 32 times for a 32-bit integer
        # Space Complexity: O(1)

        # Max positive 32-bit signed integer
        MAX = 0x7FFFFFFF
        # 32-bit mask to handle Python's arbitrary precision
        mask = 0xFFFFFFFF

        while b != 0:
            # Carry bits are computed by ANDing and shifting left by 1
            carry = (a & b) << 1
```

```

# Carry-less sum is computed by XORing and masked to 32 bits
a = (a ^ b) & mask
# Update b to hold the carry bits for the next iteration
b = carry & mask

# If the result is negative (MSB is 1, i.e., > MAX), map it back to Python's
negative representation
return a if a <= MAX else ~(a ^ mask)

```

12. [LeetCode 191: Number of 1 Bits](#) [Easy]

 **Key Points:** Bit Manipulation - Brian Kernighan's Algorithm ($n \& (n - 1)$)

Intuition & Breaking Points:

Intuition & Pitfalls: The standard solution checks the least significant bit (LSB) and shifts right 32 times. However, this always takes 32 iterations, even if the number contains only a single 1 bit.

Mathematical Derivation: Brian Kernighan's algorithm optimizes this by clearing the lowest set bit using $n \& (n - 1)$. For any number n , $n - 1$ flips all bits to the right of the lowest set bit (including the set bit itself). Thus, $n \& (n - 1)$ clears the lowest set bit while keeping all higher bits unchanged. This means the number of iterations equals exactly the number of set bits, which is extremely efficient.

```

class Solution:
    def hammingWeight(self, n: int) -> int:
        # Time Complexity: O(1) - Iterations match the number of set bits (maximum 32)
        # Space Complexity: O(1)
        count = 0
        while n:
            # Clear the lowest set bit of n
            n = n & (n - 1)

```

```
count += 1
return count
```

13. [LeetCode 338: Counting Bits](#) [Easy]

 **Key Points:** Dynamic Programming + Bit Manipulation

Intuition & Breaking Points:

Intuition & Pitfalls: Calculating the number of set bits for each number from 0 to n individually takes $O(N \log N)$ or $O(32N)$ time complexity.

Mathematical Derivation: To achieve an optimal $O(N)$ time complexity, we can use dynamic programming by reusing previously computed results. Let $dp[i]$ be the count of 1 bits in the binary representation of i . 1. Parity rules: - If i is even, $dp[i] = dp[i >> 1]$ because dividing an even number by 2 is a right shift that discards a 0 bit (no change in 1s count). - If i is odd, $dp[i] = dp[i >> 1] + 1$ because dividing an odd number by 2 is a right shift that discards a 1 bit (adds 1 to the count). 2. Unified recurrence: $dp[i] = dp[i >> 1] + (i \& 1)$. This allows us to compute each state in $O(1)$ time.

```
from typing import List

class Solution:
    def countBits(self, n: int) -> List[int]:
        # Time Complexity: O(n) - Single pass from 0 to n
        # Space Complexity: O(1) - Only the output array is allocated
        dp = [0] * (n + 1)
        for i in range(1, n + 1):
            # Recurrence: set bits in i equals set bits in i // 2 + LSB of i
            dp[i] = dp[i >> 1] + (i & 1)
        return dp
```

14. [LeetCode 268: Missing Number](#) [Easy]

 **Key Points:** Bit Manipulation (XOR) / Gauss Sum Formula

Intuition & Breaking Points:

Intuition & Pitfalls: Sorting the array and scanning takes $O(N \log N)$ time. Using a hash set takes $O(N)$ time but requires $O(N)$ auxiliary space. We need an $O(N)$ time and $O(1)$ space solution.

Mathematical Derivation: We can solve this in $O(1)$ space using two methods: 1. Gauss Sum Formula: Compute the expected sum of numbers from 0 to n using $\frac{n * (n + 1)}{2}$ and subtract the sum of the array. The difference is the missing number. (In languages like C++/Java, overflow must be prevented). 2. XOR Operation: Since $x \oplus x = 0$ and $x \oplus 0 = x$, if we XOR all indexes 0 to n and all values in the array, numbers that appear twice will cancel out to 0, leaving only the missing number that appeared once. This avoids any arithmetic overflow.

```
from typing import List

class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        # Time Complexity: O(n) - Single pass through the array
        # Space Complexity: O(1)
        missing = len(nums)
        for i, num in enumerate(nums):
            # XOR indices and numbers to cancel out matches
            missing ^= i ^ num
        return missing
```

15. [LeetCode 190: Reverse Bits](#) [Easy]

 Key Points: Bit Manipulation - Shift and Accumulate **Intuition & Breaking Points:**

Intuition & Pitfalls: We can process the 32 bits of `n` one by one. In each step, we extract the LSB of `n` and shift it to its reversed position in the result.

Mathematical Derivation: 1. Initialize the result `res = 0`. 2. Loop 32 times: - Shift the result left by 1 to make space: `res <<= 1`. - Extract the LSB of `n` and bitwise OR it into the result: `res |= (n & 1)`. - Shift `n` right by 1 to process the next bit: `n >>= 1`. This correctly reverses all 32 bits and works for both signed and unsigned representations.

```
class Solution:
    def reverseBits(self, n: int) -> int:
        # Time Complexity: O(1) - Fixed 32 iterations
        # Space Complexity: O(1)
        res = 0
        for _ in range(32):
            # Shift result left and add the LSB of n
            res = (res << 1) | (n & 1)
            # Shift n right to prepare the next bit
            n >>= 1
        return res
```


Dynamic Programming

16. [LeetCode 70: Climbing Stairs](#) [Easy]

 **Key Points:** Dynamic Programming (DP) / Fibonacci Sequence

Intuition & Breaking Points:

Intuition & Pitfalls: To reach the n -th step, your last step must be either a 1-step jump from the $(n-1)$ -th step, or a 2-step jump from the $(n-2)$ -th step. Thus, the total ways to reach the n -th step is the sum of ways to reach step $n-1$ and step $n-2$. This is the definition of the Fibonacci recurrence.

Mathematical Derivation: State transition equation: $dp[i] = dp[i-1] + dp[i-2]$ Since calculating $dp[i]$ only depends on the previous two states, we do not need to maintain a full dp array. Instead, we can use two variables `one` and `two` and scroll them forward to update, reducing the space complexity from $O(N)$ to $O(1)$.

```
class Solution:
    def climbStairs(self, n: int) -> int:
        # Time Complexity: O(n) - Single pass loop
        # Space Complexity: O(1) - Only two state variables are maintained
        if n <= 2:
            return n

        # 'one' represents dp[i-2], 'two' represents dp[i-1]
        one, two = 1, 2
        for _ in range(3, n + 1):
            temp = one + two
            one = two
            two = temp
```

```
return two
```

17. [LeetCode 322: Coin Change](#) [Medium]

 **Key Points:** Complete Knapsack Problem / Dynamic Programming

Intuition & Breaking Points:

Intuition & Pitfalls: A greedy strategy of choosing the largest denomination coin first fails for many combinations. For example, if `coins = [1, 3, 4]` and `amount = 6`, greedy selects `4 + 1 + 1` (3 coins), whereas the optimal selection is `3 + 3` (2 coins). Thus, we must evaluate all combinations using dynamic programming.

Mathematical Derivation: Let `dp[i]` be the minimum number of coins needed to make up amount `i`. To calculate `dp[i]`, we can try taking any coin of value `coin` (where `i >= coin`). If we take that coin, the remaining amount is `i - coin`. The coin count becomes `dp[i - coin] + 1`. We take the minimum over all available coins: $dp[i] = \min(dp[i], dp[i - coin] + 1)$. We initialize `dp[0] = 0` and all other entries to infinity (or `amount + 1`). Finally, we check if `dp[amount]` remains at `amount + 1` (meaning it's unreachable).

```
from typing import List

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        # Time Complexity: O(n * amount) - n is the number of coin denominations
        # Space Complexity: O(amount) - DP array size is amount + 1

        # Initialize with amount + 1 representing infinity
        dp = [amount + 1] * (amount + 1)
        dp[0] = 0 # Base case: 0 coins needed for amount 0

        for i in range(1, amount + 1):
```

```

for coin in coins:
    if i - coin >= 0:
        # Choose minimum between not using this coin and using this coin
        dp[i] = min(dp[i], dp[i - coin] + 1)

return dp[amount] if dp[amount] != amount + 1 else -1

```

18. [LeetCode 300: Longest Increasing Subsequence](#) [Medium]

 **Key Points:** Dynamic Programming $O(N^2)$ / Binary Search with Greedy $O(N \log N)$

Intuition & Breaking Points:

Intuition & Pitfalls: Let `dp[i]` be the length of the longest increasing subsequence ending at `nums[i]`. For each element `nums[i]`, we scan all preceding elements `nums[j]` (where $j < i$). If `nums[i] > nums[j]`, we can append `nums[i]` to the subsequence ending at `nums[j]`, yielding `dp[i] = max(dp[i], dp[j] + 1)`. This takes $O(N^2)$ time.

Mathematical Derivation: To achieve $O(N \log N)$ complexity, we use a greedy approach combined with binary search (Patience Sorting): We maintain an array `sub`, where `sub[i]` stores the smallest tail value of all increasing subsequences of length `i+1` found so far. For each number `x` in `nums`: - If `x` is greater than the last element of `sub`, we append `x` to `sub` (increasing the subsequence length). - Otherwise, we use binary search (`bisect_left`) to find the first element in `sub` that is greater than or equal to `x` and replace it with `x`. (Greedy choice: a smaller tail value makes it easier to append future elements). The length of `sub` is the length of the LIS.

```

from typing import List
import bisect

class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:

```

```

# Time Complexity: O(n log n) - Single pass through nums with a binary search in
each step

# Space Complexity: O(n) - Auxiliary array sub

if not nums:
    return 0

sub = []

for x in nums:
    # Find the index of the first element >= x in sub
    idx = bisect.bisect_left(sub, x)
    if idx == len(sub):
        sub.append(x) # Append if x is larger than all elements in sub
    else:
        sub[idx] = x # Replace the element at idx with x

return len(sub)

```

19. [LeetCode 1143: Longest Common Subsequence](#) [Medium]

 **Key Points:** 2D Dynamic Programming (2D DP)

Intuition & Breaking Points:

Intuition & Pitfalls: To find the longest common subsequence of `text1` and `text2`, we can compare characters from the ends. If they match, they can be part of the LCS. If they don't, we can try skipping the last character of either `text1` or `text2` and take the maximum.

Mathematical Derivation: Let `dp[i][j]` be the length of the LCS of `text1[0...i-1]` and `text2[0...j-1]`. State transition: 1. If `text1[i - 1] == text2[j - 1]`, the characters match: `dp[i][j] = dp[i - 1][j - 1] + 1`. 2. If `text1[i - 1] != text2[j - 1]`, they do not match: `dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])`. We create a 2D matrix of size `(m+1) \* (n+1)` initialized to 0. We fill the matrix bottom-up, and `dp[m][n]` yields the result.

```

class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        # Time Complexity: O(m * n) - Fill the 2D DP matrix
        # Space Complexity: O(m * n) - Size of the 2D DP matrix
        m, n = len(text1), len(text2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    # Match found: diagonal value + 1
                    dp[i][j] = dp[i - 1][j - 1] + 1
                else:
                    # No match: take max of left and top values
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

        return dp[m][n]

```

20. [LeetCode 139: Word Break](#) [Medium]

 **Key Points:** Dynamic Programming - Substring Checking via Hash Set

Intuition & Breaking Points:

Intuition & Pitfalls: A naive DFS recursively partitions `s` by matching words from `wordDict`. However, without memoization, this yields exponential time complexity due to redundant subproblem checks. We must use dynamic programming.

Mathematical Derivation: Let `dp[i]` be a boolean indicating whether the prefix `s[0...i-1]` can be segmented into words from `wordDict`. State transition: To calculate `dp[i]`, we check all partitioning splits `j` (where `0 <= j < i`). If `dp[j]` is True (meaning prefix `s[0...j-1]` is valid) and the suffix `s[j...i-1]` is present in the dictionary, then `dp[i]` becomes True. Formula: `dp[i] = any(dp[j] and s[j:i] in wordSet for j in range(i))`. Base case: `dp[0] = True` (empty string is always valid). Output is `dp[len(s)]`.

```

from typing import List

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        # Time Complexity: O(n^2) - Outer loop n steps, inner loop checks up to n divisions
        # Space Complexity: O(n + m) - DP array size n, wordSet size m
        word_set = set(wordDict) # O(1) average lookup time
        dp = [False] * (len(s) + 1)
        dp[0] = True # Base case: empty string is segmentable

        for i in range(1, len(s) + 1):
            for j in range(i):
                # If prefix s[:j] is valid and suffix s[j:i] is in dict
                if dp[j] and s[j:i] in word_set:
                    dp[i] = True
                    break # Found a valid partition, skip further checks for index i

        return dp[len(s)]

```

21. [LeetCode 377: Combination Sum IV](#) [Medium]

 **Key Points:** Unbounded Knapsack Permutations / Dynamic Programming

Intuition & Breaking Points:

Intuition & Pitfalls: This is similar to the Coin Change problem but asks for the number of combinations (permutations where different orders count as distinct). For example, if `nums = [1, 2, 3]` and `target = 4`, `(1, 3)` and `(3, 1)` are distinct. Because order matters, we iterate over the targets in the outer loop, and over the numbers in the inner loop.

Mathematical Derivation: Let `dp[i]` be the number of permutations that sum to `i`. To make a sum of `i`, we can choose any number `num` from `nums` (where `i >= num`) as the last element. The number of permutations is then the sum of `dp[i - num]`.

num]` for all choices of `num`: `dp[i] = sum(dp[i - num] for num in nums)` Base case: `dp[0] = 1` (one way to form sum 0, i.e., empty set).

```
from typing import List

class Solution:
    def combinationSum4(self, nums: List[int], target: int) -> int:
        # Time Complexity: O(target * n) - target is the target sum, n is the size of nums
        # Space Complexity: O(target)
        dp = [0] * (target + 1)
        dp[0] = 1 # Base case: one way to make sum 0

        for i in range(1, target + 1):
            for num in nums:
                if i - num >= 0:
                    dp[i] += dp[i - num]

        return dp[target]
```

22. [LeetCode 198: House Robber](#) [Medium]

 **Key Points:** Dynamic Programming - Space Optimization

Intuition & Breaking Points:

Intuition & Pitfalls: When robbing houses, we cannot rob adjacent ones. At house `i`, we have two options: 1. Rob this house, meaning we cannot rob house `i-1`. The max profit is `dp[i-2] + nums[i]`. 2. Skip this house, meaning we keep the max profit from house `i-1`, which is `dp[i-1]`. We choose the maximum of these two options.

Mathematical Derivation: Let `dp[i]` be the max amount we can rob from the first `i` houses. Recurrence relation: `dp[i] = max(dp[i-1], dp[i-2] + nums[i])` Space Optimization: Since `dp[i]` only depends on `dp[i-1]` and `dp[i-2]`, we can use two

variables `rob1` (representing `dp[i-2]`) and `rob2` (representing `dp[i-1]`) to scroll forward, optimizing space from $O(N)$ to $O(1)$.

```
from typing import List

class Solution:
    def rob(self, nums: List[int]) -> int:
        # Time Complexity:  $O(N)$  - Single pass loop
        # Space Complexity:  $O(1)$  - Two variables scroll state

        # rob1 acts as dp[i-2], rob2 acts as dp[i-1]
        rob1, rob2 = 0, 0

        for num in nums:
            # Choose between robbing current house + rob1, or skipping it (rob2)
            temp = max(rob1 + num, rob2)
            rob1 = rob2
            rob2 = temp

        return rob2
```

23. [LeetCode 213: House Robber II](#) [Medium]

 **Key Points:** Circular Array Dynamic Programming

Intuition & Breaking Points:

Intuition & Pitfalls: The only difference from House Robber I is that the houses are arranged in a circle, meaning the first house and the last house are adjacent and cannot be robbed together.

Mathematical Derivation: To break this circular constraint, we can split the problem into two linear subproblems: 1. If we rob the first house, we cannot rob the last house. The problem simplifies to a linear scan of houses `[0...n-2]`. 2. If we skip the first house, we can potentially rob the last house. The problem simplifies to a linear

scan of houses `[1...n-1]`. Since the optimal solution must fall in one of these two categories, we run the linear solver on both sub-arrays and return the maximum of the two. Base case: if there is only 1 house, return its value directly.

```
from typing import List

class Solution:
    def rob(self, nums: List[int]) -> int:
        # Time Complexity: O(N) - Linear scans of arrays of size N-1
        # Space Complexity: O(1)
        if len(nums) == 1:
            return nums[0]

        # Helper to compute linear house robber
        def rob_linear(house_prices: List[int]) -> int:
            rob1, rob2 = 0, 0
            for price in house_prices:
                temp = max(rob1 + price, rob2)
                rob1 = rob2
                rob2 = temp
            return rob2

        # Take the maximum of skipping the last house, and skipping the first house
        return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))
```

24. [LeetCode 91: Decode Ways](#) [Medium]

 **Key Points:** Dynamic Programming - Split Checking with Leading Zeroes

Intuition & Breaking Points:

Intuition & Pitfalls: We want to decode a string of digits `s` where each mapping corresponds to a character (1-26). Similar to climbing stairs, at each step we can

decode either 1 digit or 2 digits, but we must validate the ranges (e.g., '0' is invalid on its own, '30' is invalid).

Mathematical Derivation: Let $dp[i]$ be the number of decode ways for prefix $s[0...i-1]$. Recurrence relation: 1. If $s[i-1]$ is not '0', it can be decoded as a single digit: $dp[i] += dp[i-1]$. 2. If the two-digit substring $s[i-2:i]$ forms a valid number between '10' and '26', it can be decoded as a double digit: $dp[i] += dp[i-2]$. Base cases: $dp[0] = 1$. If the string starts with '0', return 0. We can optimize the space to $O(1)$ using rolling variables.

```
class Solution:
    def numDecodings(self, s: str) -> int:
        # Time Complexity: O(n) - Single pass loop
        # Space Complexity: O(1) - Two variables for state tracking
        if not s or s[0] == '0':
            return 0

        # prev2 acts as dp[i-2], prev1 acts as dp[i-1]
        prev2, prev1 = 1, 1

        for i in range(1, len(s)):
            current = 0
            # 1. Decode as single digit
            if s[i] != '0':
                current += prev1
            # 2. Decode as double digit
            two_digit = int(s[i-1:i+1])
            if 10 <= two_digit <= 26:
                current += prev2

            prev2 = prev1
            prev1 = current

        return prev1
```

25. [LeetCode 62: Unique Paths](#) [Medium]

 **Key Points:** Grid Dynamic Programming / Space Compression

Intuition & Breaking Points:

Intuition & Pitfalls: Since the robot can only move down or right, any cell (i, j) can only be reached from the cell above $(i-1, j)$ or the cell to the left $(i, j-1)$. The number of unique paths to (i, j) is the sum of paths to these two cells.

Mathematical Derivation: Let $dp[i][j]$ be the number of unique paths to grid cell (i, j) . Recurrence relation: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. The first row and column are initialized to 1 because there is only one way to go straight down or right. Space Optimization: Since the current row only depends on the row above and the current row's left cell, we can compress the space to a 1D array of size n and update it iteratively via $row[j] = row[j] + row[j-1]$.

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # Time Complexity: O(m * n) - Single loop over all grid cells
        # Space Complexity: O(n) - Single row array to accumulate states

        # Initialize paths in the first row to 1
        row = [1] * n

        # Update row by row
        for _ in range(m - 1):
            new_row = [1] * n
            for j in range(1, n):
                # new_row[j] (left cell) + row[j] (cell from row above)
                new_row[j] = row[j] + new_row[j-1]
            row = new_row

        return row[n-1]
```

26. [LeetCode 55: Jump Game](#) [Medium]

 **Key Points:** Greedy Pointer Shifting

Intuition & Breaking Points:

Intuition & Pitfalls: Let `dp[i]` be whether position `i` can reach the end. However, a greedy strategy traversing backwards from the goal is much more efficient and uses less space.

Mathematical Derivation: Greedy approach (backwards): We define our destination goal `goal` as `n-1` (the last index of the array). We traverse the array from right to left. If from position `i` we can jump to or beyond the current `goal` (i.e., `i + nums[i] >= goal`), then as long as we can reach `i`, we can reach `goal`. We then shift our `goal` to `i` (`goal = i`). After checking the whole array, if `goal` successfully shifts back to `0`, we can reach the end from the start in $O(N)$ time and $O(1)$ space.

```
from typing import List

class Solution:
    def canJump(self, nums: List[int]) -> bool:
        # Time Complexity: O(N) - Single pass from right to left
        # Space Complexity: O(1)

        # Goal is initialized to the last index of the array
        goal = len(nums) - 1

        for i in range(len(nums) - 1, -1, -1):
            # If current index + jump reach can hit or exceed the goal
            if i + nums[i] >= goal:
                # Update goal to be the current index
                goal = i

        # If goal successfully rolled back to index 0, we can reach the end
        return goal == 0
```

Graphs

27. [LeetCode 133: Clone Graph](#) [Medium]

 **Key Points:** Graph Traversals - DFS / BFS with Hash Map Mapping

Intuition & Breaking Points:

Intuition & Pitfalls: To clone a graph, we must traverse all of its nodes. However, because graphs can contain cycles, we must keep track of nodes we have already copied to prevent falling into infinite recursion.

Mathematical Derivation: We use a hash map `cloned` to store the mapping from original nodes to their cloned counterparts (`old\_node` -> `new\_node`). For each visited node: 1. If it is already in `cloned`, return the cloned node directly. 2. Otherwise, instantiate a new node with the same value and save it in `cloned`. 3. Recursively copy all its neighbors, adding each cloned neighbor to the new node's neighbor list. This DFS + Hash Map pattern ensures every node and edge is copied once without cycles.

```
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        # Time Complexity: O(V + E) - Each node and edge is visited once
        # Space Complexity: O(V) - Recursion stack and cloned hash map
        if not node:
            return None

        cloned = {}
```

```
def dfs(curr):  
    # If already cloned, return the cached clone to break cycles  
    if curr in cloned:  
        return cloned[curr]  
  
    # Instantiate the copy and cache it immediately  
    copy = Node(curr.val)  
    cloned[curr] = copy  
  
    # Recursively copy all neighbor nodes  
    for neighbor in curr.neighbors:  
        copy.neighbors.append(dfs(neighbor))  
  
    return copy  
  
return dfs(node)
```

28. [LeetCode 207: Course Schedule](#) [Medium]

 **Key Points:** Topological Sort - Kahn's Algorithm (BFS) / DFS Cycle Detection

Intuition & Breaking Points:

Intuition & Pitfalls: The course prerequisite dependencies can be modeled as a directed graph where an edge from `A` to `B` means course `A` is a prerequisite for course `B`. To finish all courses, this directed graph must be a Directed Acyclic Graph (DAG) containing no cycles.

Mathematical Derivation: A standard way to detect cycles in a directed graph is Topological Sort. We use Kahn's Algorithm (BFS-based): 1. Build an adjacency list and compute the in-degrees (number of incoming edges) for all nodes. 2. Add all nodes with an in-degree of 0 (courses with no prerequisites) to a queue. 3. While the queue is not empty, dequeue a node, increment the count of visited courses, and iterate through its neighbors. For each neighbor, decrement its in-degree by 1. If its in-degree drops to 0, enqueue it. 4. Finally, check if the visited count equals the total

number of courses `numCourses`. If it does, a valid topological sort exists (no cycles); otherwise, there is a cycle.

```
from typing import List
from collections import deque

class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        # Time Complexity:  $O(V + E)$  -  $V$  is the number of courses,  $E$  is the number of
        # prerequisites
        # Space Complexity:  $O(V + E)$  - Adjacency list and in-degree table

        # Build adjacency list and in-degree array
        adj = {i: [] for i in range(numCourses)}
        indegree = [0] * numCourses

        for course, pre in prerequisites:
            adj[pre].append(course)
            indegree[course] += 1

        # Dequeue starting courses with 0 prerequisites (in-degree = 0)
        queue = deque([i for i in range(numCourses) if indegree[i] == 0])
        visited_count = 0

        while queue:
            curr = queue.popleft()
            visited_count += 1

            # Decrement in-degree for all depending neighbors
            for neighbor in adj[curr]:
                indegree[neighbor] -= 1
                # Enqueue neighbor if all its prerequisites are completed
                if indegree[neighbor] == 0:
                    queue.append(neighbor)

        return visited_count == numCourses
```

29. [LeetCode 417: Pacific Atlantic Water Flow](#) [Medium]

 **Key Points:** Multi-Source DFS / Reverse Search

Intuition & Breaking Points:

Intuition & Pitfalls: We need to find grid coordinates that can flow to both the Pacific Ocean (top/left) and the Atlantic Ocean (bottom/right). Running a separate flow search from every cell would take $O((M * N)^2)$ time, which is too slow.

Mathematical Derivation: Reverse thinking: Instead of starting from each grid cell and searching downwards to the oceans, we start from the ocean boundaries and search backwards (climbing up to adjacent cells with height $\geq$ current height). 1. Declare two boolean matrices: `pacific\_reach` and `atlantic\_reach` to store reachable coordinates from the respective ocean. 2. Run DFS from the Pacific boundaries (first row and first column) to mark all reachable cells. 3. Run DFS from the Atlantic boundaries (last row and last column) to mark all reachable cells. 4. Traverse the grid. Any cell marked True in both matrices can flow to both oceans. This reduces the time complexity to $O(M * N)$.

```
from typing import List

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        # Time Complexity: O(m * n) - Each cell is visited a constant number of times
        # Space Complexity: O(m * n) - Reachability grids and recursive DFS stack
        if not heights or not heights[0]:
            return []

        m, n = len(heights), len(heights[0])
        pacific_reach = [[False] * n for _ in range(m)]
        atlantic_reach = [[False] * n for _ in range(m)]

        def dfs(r, c, reach, prev_height):
            # Base cases: out of bounds, already visited, or water cannot flow backwards
            (height < prev_height)

            if (r < 0 or r >= m or c < 0 or c >= n or
```



```

        reach[r][c] or heights[r][c] < prev_height):
            return
        reach[r][c] = True

        # Explore 4 directions
        dfs(r + 1, c, reach, heights[r][c])
        dfs(r - 1, c, reach, heights[r][c])
        dfs(r, c + 1, reach, heights[r][c])
        dfs(r, c - 1, reach, heights[r][c])

    # Run DFS from the Pacific and Atlantic boundaries
    for i in range(m):
        dfs(i, 0, pacific_reach, heights[i][0])
        dfs(i, n - 1, atlantic_reach, heights[i][n - 1])
    for j in range(n):
        dfs(0, j, pacific_reach, heights[0][j])
        dfs(m - 1, j, atlantic_reach, heights[m - 1][j])

    # Find intersections of Pacific and Atlantic reachability
    res = []
    for r in range(m):
        for c in range(n):
            if pacific_reach[r][c] and atlantic_reach[r][c]:
                res.append([r, c])

    return res

```

30. [LeetCode 200: Number of Islands](#) [Medium]

 **Key Points:** Grid Search - DFS / BFS / Union-Find

Intuition & Breaking Points:

Intuition & Pitfalls: We need to find the number of connected components of '1's (land) surrounded by '0's (water). When we encounter a '1', it marks the discovery of a new island. We must then run DFS/BFS to traverse and sink all connected lands to avoid double counting.

Mathematical Derivation: 1. Iterate through every cell `(r, c)` of the grid. 2. If `grid[r][c] == '1'`: - Increment the island counter. - Start DFS from `(r, c)`. For each adjacent land cell, we sink it by setting its value to `0` (in-place modification is a great space optimization that avoids a `visited` set). 3. Continue scanning the grid; any future `1` encountered must belong to a separate, new island.

```
from typing import List

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        # Time Complexity: O(m * n) - Each grid cell is visited a constant number of times
        # Space Complexity: O(m * n) - Recursion stack in the worst case (all land)
        if not grid:
            return 0

        m, n = len(grid), len(grid[0])
        island_count = 0

        def dfs(r, c):
            # Base cases: out of bounds or water cell
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] == '0':
                return

            # Sink current land to prevent visiting it again
            grid[r][c] = '0'

            # Recursively explore neighbors
            dfs(r + 1, c)
            dfs(r - 1, c)
            dfs(r, c + 1)
            dfs(r, c - 1)

        for r in range(m):
            for c in range(n):
                if grid[r][c] == '1':
                    island_count += 1
                    # Initiate DFS to sink all connected parts of the island
                    dfs(r, c)

        return island_count
```

31. [LeetCode 128: Longest Consecutive Sequence](#) [Medium]

 **Key Points:** Hash Set - $O(N)$ Time Optimization

Intuition & Breaking Points:

Intuition & Pitfalls: Sorting the array and scanning takes $O(N \log N)$ time complexity. We need to find an $O(N)$ solution using extra memory.

Mathematical Derivation: To achieve $O(N)$ time complexity, we can use a hash set for $O(1)$ membership lookups: 1. Insert all numbers into a hash set `num\_set`. 2. Iterate through each number `num` in the set: - Check if `num - 1` is in the set. If it is not, then `num` is the starting element of a consecutive sequence. - From this starting element, increment and check for `num + 1`, `num + 2`... in the set, counting the length of this sequence. - Update the global maximum length. Since a sequence is only checked from its absolute starting element, each number is processed at most twice, yielding a strictly linear $O(N)$ time complexity.

```
from typing import List

class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        # Time Complexity:  $O(N)$  - Each number is processed at most twice
        # Space Complexity:  $O(N)$  - Storage for the hash set
        num_set = set(nums)
        longest_streak = 0

        for num in num_set:
            # Check if num is the start of a sequence
            if num - 1 not in num_set:
                current_num = num
                current_streak = 1

                # Check for consecutive elements
                while current_num + 1 in num_set:
                    current_num += 1
                    current_streak += 1
```

```

        longest_streak = max(longest_streak, current_streak)

    return longest_streak

```

32. [LeetCode 269: Alien Dictionary](#) [Hard]

 **Key Points:** Topological Sort / Cycle Detection / Directed Graphs

Intuition & Breaking Points:

Intuition & Pitfalls: Given a sorted list of words in an alien language, we need to extract the order of characters. In a dictionary, word ordering is determined by the first differing character. We can build directed edges `c1 -> c2` by comparing adjacent words and then run a topological sort.

Mathematical Derivation: 1. **Graph Construction**: Compare adjacent words `w1` and `w2` character by character. The first mismatch `w1[j] != w2[j]` gives an edge `w1[j] -> w2[j]`. If `w1` is longer than `w2` and `w2` is a prefix of `w1` (e.g., 'abc' comes before 'ab'), this is an invalid order, so we return "". 2. **Topological Sort (DFS / BFS)**: Run DFS with three-state coloring (0: unvisited, 1: visiting, 2: visited) to detect cycles. If we encounter a node in state 1 during DFS, there is a cycle, so we return "". If the graph is acyclic, reverse the DFS finish order to get the topological order.

```

from typing import List

class Solution:
    def alienOrder(self, words: List[str]) -> str:
        # Time Complexity: O(C) - C is the total length of all words combined
        # Space Complexity: O(V + E) - Adjacency list representation (V <= 26)

        # Initialize graph adjacency list
        adj = {char: set() for word in words for char in word}

```

```
# 1. Build directed graph by comparing adjacent words
for i in range(len(words) - 1):
    w1, w2 = words[i], words[i + 1]
    min_len = min(len(w1), len(w2))
    # Prefix boundary check: if w1 is longer and has w2 as prefix, order is invalid
    if len(w1) > len(w2) and w1[:min_len] == w2:
        return ""
    for j in range(min_len):
        if w1[j] != w2[j]:
            adj[w1[j]].add(w2[j])
            break

# 2. Cycle detection and topological sort using DFS
visiting = set() # Set of nodes currently in recursion stack
visited = set() # Set of nodes fully processed
res = []

def dfs(char):
    if char in visiting:
        return False # Cycle detected
    if char in visited:
        return True

    visiting.add(char)
    for neighbor in adj[char]:
        if not dfs(neighbor):
            return False
    visiting.remove(char)
    visited.add(char)
    res.append(char) # Append node after exploring all paths from it
    return True

for char in adj:
    if not dfs(char):
        return "" # Cycle exists, return empty

# Reverse the result list since DFS returns reverse order
return "".join(res[::-1])
```

33. [LeetCode 261: Graph Valid Tree](#) [Medium]

 **Key Points:** Undirected Graph - Cycle Detection & Connectivity / Union-Find

Intuition & Breaking Points:

Intuition & Pitfalls: An undirected graph is a tree if and only if it is **connected** and **acyclic**. In graph theory, a graph with n nodes is a tree if: 1. It has exactly $n - 1$ edges. 2. It contains no cycles.

Mathematical Derivation: We can use Union-Find (Disjoint Set) to check this efficiently: 1. If $\text{len}(\text{edges}) \neq n - 1$, the graph cannot be a tree, return False. 2. Initialize a disjoint set. Iterate through each edge (u, v) and try to union their sets: - If $\text{find}(u) == \text{find}(v)$ before union, u and v are already connected, so adding this edge creates a cycle. Return False. - Otherwise, union their sets. 3. Since we have verified $\text{edges} == n - 1$ and confirmed no cycles, the graph is guaranteed to be fully connected. Return True.

```
from typing import List

class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        # Time Complexity: O(N * alpha(N)) - alpha is the inverse Ackermann function,
        # roughly O(1)
        # Space Complexity: O(N) - Disjoint Set parent array

        # Necessary condition: tree must have exactly n - 1 edges
        if len(edges) != n - 1:
            return False

        parent = [i for i in range(n)]

        def find(i):
            if parent[i] == i:
                return i

            # Path compression optimization
            parent[i] = find(parent[i])
```

```
        return parent[i]

    def union(i, j):
        root_i = find(i)
        root_j = find(j)
        # If roots are same, a cycle is detected
        if root_i == root_j:
            return False
        parent[root_i] = root_j
        return True

    for u, v in edges:
        if not union(u, v):
            return False # Cycle detected

    return True
```

34. [LeetCode 323: Number of Connected Components in an Undirected Graph](#) [Medium]

 **Key Points:** Connected Components / Union-Find

Intuition & Breaking Points:

Intuition & Pitfalls: Given an undirected graph, we need to find the number of disconnected subgraphs. While DFS/BFS traversal works, Union-Find provides a highly concise and efficient disjoint set union solution.

Mathematical Derivation: 1. Initially, assume all n nodes are isolated components ($\text{count} = n$). 2. Iterate through each edge (u, v) : - Find the roots of u and v in the disjoint set. - If the roots are different, they belong to different components. We merge (union) the sets and decrement count by 1. - If they have the same root, they are already connected, so we do nothing. 3. The final value of count is the number of connected components.

```
from typing import List

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        # Time Complexity:  $O(N + E * \alpha(N))$  - N nodes, E edges. Virtually  $O(N + E)$ 
        # Space Complexity:  $O(N)$  - Disjoint Set parent array
        parent = [i for i in range(n)]
        count = n

        def find(i):
            if parent[i] == i:
                return i
            # Path compression
            parent[i] = find(parent[i])
            return parent[i]

        def union(i, j):
            nonlocal count
            root_i = find(i)
            root_j = find(j)
            if root_i != root_j:
                # Merge the components
                parent[root_i] = root_j
                # Decrement component count upon successful merge
                count -= 1

        for u, v in edges:
            union(u, v)

        return count
```


Intervals

35. [LeetCode 57: Insert Interval](#) [Medium]

 **Key Points:** Interval Traversal & Merging - Three-Stage Partitioning

Intuition & Breaking Points:

Intuition & Pitfalls: We need to insert a new interval into a list of sorted, non-overlapping intervals, ensuring the list remains sorted and non-overlapping. Directly inserting the interval and sorting the entire list ruins the pre-sorted property, taking $O(N \log N)$ time.

Mathematical Derivation: To solve this in linear $O(N)$ time, we can partition the intervals into three stages during a single pass: 1. **Before the overlap**: Add all intervals that end before the new interval starts (`interval[1] < newInterval[0]`). These can be appended to the result directly. 2. **Overlap merging**: For any interval that overlaps with the new interval (satisfying `interval[0] <= newInterval[1]` and `interval[1] >= newInterval[0]`), merge it into the new interval by taking the minimum start time `min(interval[0], newInterval[0])` and maximum end time `max(interval[1], newInterval[1])`. This is an iterative update. 3. **After the overlap**: Add all remaining intervals that start after the merged new interval ends (`interval[0] > newInterval[1]`).

```
from typing import List

class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        # Time Complexity: O(N) - Single pass through the intervals list
        # Space Complexity: O(1) - Modifying output array directly (excluding output
        # allocation)
        res = []
        i = 0
```

```
n = len(intervals)

# 1. Add all intervals that end before the new interval starts
while i < n and intervals[i][1] < newInterval[0]:
    res.append(intervals[i])
    i += 1

# 2. Merge all overlapping intervals with newInterval
while i < n and intervals[i][0] <= newInterval[1]:
    newInterval[0] = min(newInterval[0], intervals[i][0])
    newInterval[1] = max(newInterval[1], intervals[i][1])
    i += 1
res.append(newInterval)

# 3. Add all remaining intervals that start after newInterval ends
while i < n:
    res.append(intervals[i])
    i += 1

return res
```

36. [LeetCode 56: Merge Intervals](#) [Medium]

 **Key Points:** Sorting + Single Pass Merging

Intuition & Breaking Points:

Intuition & Pitfalls: If the intervals are unsorted, we cannot easily determine which ones overlap. By sorting the intervals by their start times, any potentially overlapping intervals will be grouped adjacent to each other.

Mathematical Derivation: 1. Sort the intervals by their start times:

``intervals.sort(key=lambda x: x[0])``. 2. Initialize a list ``merged`` with the first sorted interval. 3. For each subsequent interval ``curr``: - Retrieve the last merged interval ``prev`` from ``merged``. - **Overlapping condition**: If ``curr[0] <= prev[1]``, the current interval starts before or at the end of the previous interval. Since the list is sorted,

`curr[0] >= prev[0]` is guaranteed. We simply update the end time of `prev` to `max(prev[1], curr[1])`. - **Non-overlapping condition**: If `curr[0] > prev[1]`, append `curr` as a new independent interval in `merged`.

```
from typing import List

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        # Time Complexity: O(N log N) - Sorting dominates the time complexity; linear
        # traversal is O(N)
        # Space Complexity: O(log N) or O(N) - Auxiliary space required by sorting
        # algorithms
        if not intervals:
            return []

        # Sort intervals by their start times in ascending order
        intervals.sort(key=lambda x: x[0])

        merged = [intervals[0]]
        for curr in intervals[1:]:
            prev = merged[-1]
            # If the current interval overlaps with the previous one, merge them
            if curr[0] <= prev[1]:
                prev[1] = max(prev[1], curr[1])
            else:
                # Otherwise, append current interval as a new entry
                merged.append(curr)

        return merged
```

37. [LeetCode 435: Non-overlapping Intervals](#) [Medium]

 **Key Points:** Greedy Algorithm - Interval Scheduling Problem

💡 Intuition & Breaking Points:

Intuition & Pitfalls: We need to remove the minimum number of intervals to eliminate all overlaps. This is mathematically equivalent to: ****maximizing the number of non-overlapping intervals**** we can keep. The greedy approach is optimal for this class of interval scheduling problems.

Mathematical Derivation: Greedy choice: To leave as much room as possible for future intervals, we should ****always select the interval that ends earliest****. 1. Sort the intervals in ascending order by their ****end times****. 2. Maintain `end`` representing the end time of the last chosen interval, initialized to the first interval's end time. Maintain a `count`` initialized to 0 representing removed intervals. 3. Iterate through subsequent intervals `curr``: - If `curr[0] >= end``, it starts after the previous interval ends. We can safely keep it, so we update `end = curr[1]`. - If `curr[0] < end``, there is an overlap. Based on our greedy choice, we discard `curr`` (since its end time is `>= end``, keeping it would only shrink future scheduling space) and increment `count`` by 1.

```
from typing import List

class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        # Time Complexity: O(N log N) - Sorting takes O(N log N); single pass scan takes O(N)
        # Space Complexity: O(log N) - Space complexity of the sorting algorithm
        if not intervals:
            return 0

        # Greedy rule: Sort intervals by end times
        intervals.sort(key=lambda x: x[1])

        count = 0
        end = intervals[0][1] # Track end time of the last scheduled interval

        for i in range(1, len(intervals)):
            # If current interval starts before last ended, it overlaps. Remove it.
            if intervals[i][0] < end:
                count += 1
            else:
```

```

        # Otherwise, schedule it and update the end boundary
        end = intervals[i][1]

    return count

```

38. [LeetCode 252: Meeting Rooms](#) [Easy]

 **Key Points:** Sorting + Overlap Check

Intuition & Breaking Points:

Intuition & Pitfalls: For a person to attend all meetings, there must be no overlapping intervals. We should sort the meetings chronologically and check if any adjacent meetings conflict.

Mathematical Derivation: 1. Sort the list of meetings by their start times in ascending order. 2. Traverse the sorted meetings, comparing adjacent meetings $i-1$ and i : - If the start time of meeting i is less than the end time of meeting $i-1$ ($\text{intervals}[i][0] < \text{intervals}[i-1][1]$), they overlap and conflict, so we return False. 3. If the loop completes with no conflicts, return True.

```

from typing import List

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        # Time Complexity: O(N log N) - Sorting dominates the run time
        # Space Complexity: O(log N) - Auxiliary space required by sorting

        # Sort meetings by start times
        intervals.sort(key=lambda x: x[0])

        for i in range(1, len(intervals)):
            # If a meeting starts before the previous one ends, return False
            if intervals[i][0] < intervals[i-1][1]:
                return False

```

```
return True
```

39. [LeetCode 253: Meeting Rooms II](#) [Medium]

 **Key Points:** Min-Heap / Two Pointers / Chronological Sweep-Line

Intuition & Breaking Points:

Intuition & Pitfalls: We need to find the maximum number of concurrent meetings running at any point in time. This maximum concurrency determines the minimum number of meeting rooms required.

Mathematical Derivation: Chronological Sweep-Line approach: We separate the start times and end times of all meetings into two distinct arrays and sort them individually. 1. `start\_times` stores all meeting start times and is sorted. 2. `end\_times` stores all meeting end times and is sorted. 3. Place two pointers `s` and `e` at the start of these arrays. Maintain `curr\_rooms` (active rooms at current timestamp) and `max\_rooms` (the global peak value). 4. Advance through time: - If `start\_times[s] < end\_times[e]`, it means a new meeting starts before the earliest active meeting ends. We must allocate a new room: `curr\_rooms += 1`, `s += 1`. - If `start\_times[s] >= end\_times[e]`, an active meeting ends. A room is freed: `curr\_rooms -= 1`, `e += 1`. - At each step, update `max\_rooms = max(max\_rooms, curr\_rooms)`. This sweep-line approach is $O(N \log N)$ and simpler than using a min-heap.

```
from typing import List

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        # Time Complexity: O(N log N) - Sorting start and end times dominates the run time
        # Space Complexity: O(N) - Storage for start and end time arrays
        if not intervals:
            return 0
```

```
# Extract and sort start times and end times independently
start_times = sorted([i[0] for i in intervals])
end_times = sorted([i[1] for i in intervals])

s, e = 0, 0
curr_rooms = 0
max_rooms = 0

while s < len(intervals):
    # If a new meeting starts before the earliest active meeting ends
    if start_times[s] < end_times[e]:
        curr_rooms += 1
        s += 1
    else:
        # Otherwise, a meeting ended, releasing a room
        curr_rooms -= 1
        e += 1
    max_rooms = max(max_rooms, curr_rooms)

return max_rooms
```

Linked Lists

40. [LeetCode 206: Reverse Linked List](#) [Easy]

 **Key Points:** Linked List Basics - Two Pointers Iterative

Intuition & Breaking Points:

Intuition & Pitfalls: To reverse a singly linked list, we need to alter the direction of each node's pointer. Specifically, we change the `next` pointer of each node to point to its predecessor `prev`. Since it's a singly linked list, changing the `next` pointer causes us to lose the reference to the rest of the list. Thus, we must temporarily store the next node before modifying the current pointer.

Mathematical Derivation: 1. Initialize two pointers: `curr` pointing to the head node, and `prev` initialized to `None` (as the new tail should point to null). 2. Traverse the list: - **Preserve successor**: Store the next node: `nxt = curr.next`. - **Reverse link**: Point the current node back to the predecessor: `curr.next = prev`. - **Advance pointers**: Shift `prev` to `curr` (`prev = curr`) and `curr` to the stored successor (`curr = nxt`). 3. When `curr` becomes null, the traversal is complete, and `prev` points to the new head of the reversed list. Return `prev`.

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # Time Complexity: O(N) - Single pass through the linked list
```



```
# Space Complexity: O(1) - Constant pointer variables

prev = None
curr = head

while curr:
    nxt = curr.next    # Temporarily store the next node to prevent disconnection
    curr.next = prev   # Reverse current node's pointer
    prev = curr        # Shift prev pointer forward
    curr = nxt         # Shift curr pointer forward

return prev
```

41. [LeetCode 141: Linked List Cycle](#) [Easy]

 **Key Points:** Two Pointers - Fast & Slow Pointers (Floyd's Cycle-Finding Algorithm)

Intuition & Breaking Points:

Intuition & Pitfalls: To detect if a linked list contains a cycle, we could store visited nodes in a hash set. If we encounter a node already in the set, a cycle exists. However, this takes $O(N)$ auxiliary space. Can we solve this in $O(1)$ space?

Mathematical Derivation: Floyd's Cycle-Finding Algorithm (Tortoise and Hare): Initialize two pointers at the head: - Slow pointer `slow` moves 1 step at a time: `slow = slow.next`. - Fast pointer `fast` moves 2 steps at a time: `fast = fast.next.next`. If the list is acyclic, `fast` will eventually reach the end of the list (null), and we can return False. If the list contains a cycle, both pointers will eventually enter the loop. Since `fast` is moving twice as fast as `slow`, the relative distance between them decreases by 1 in each step. Consequently, `fast` will eventually catch up to (or lap) `slow` inside the loop (i.e., `slow == fast`), at which point we return True.

```
from typing import Optional

class ListNode:
```

```

def __init__(self, val=0, next=None):
    self.val = val
    self.next = next

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        # Time Complexity: O(N) - Slow pointer moves at most N steps
        # Space Complexity: O(1) - Only two pointers are allocated
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next          # Slow pointer moves 1 step
            fast = fast.next.next     # Fast pointer moves 2 steps

            # If pointers meet, a cycle exists
            if slow == fast:
                return True

        return False # Fast pointer reached the end of the list, no cycle

```

42. [LeetCode 21: Merge Two Sorted Lists](#) [Easy]

 **Key Points:** Linked List Merging / Dummy Node Technique

Intuition & Breaking Points:

Intuition & Pitfalls: To merge two sorted lists, we compare their heads and connect the smaller node to the merged list, advancing that list's pointer. This is similar to the merge step in Merge Sort.

Mathematical Derivation: To avoid complex edge cases when initializing the head of the merged list, we use a **Dummy Node** `dummy`: 1. Initialize `dummy = ListNode(0)` and a traversal pointer `curr = dummy`. 2. While both `list1` and `list2` are not null: - Compare `list1.val` and `list2.val`. - Connect `curr.next` to the node with the smaller value, then advance that list's pointer. - Advance `curr`. 3. Once the loop

terminates, at least one list is exhausted. We append the remaining elements of the non-empty list directly to `curr.next` since they are already sorted. 4. Return `dummy.next`.

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        # Time Complexity: O(N + M) - N and M are the lengths of the two lists
        # Space Complexity: O(1) - Merging is done in-place by splicing pointers
        dummy = ListNode(0) # Dummy node to simplify edge cases
        curr = dummy

        while list1 and list2:
            if list1.val <= list2.val:
                curr.next = list1
                list1 = list1.next
            else:
                curr.next = list2
                list2 = list2.next
            curr = curr.next

        # Append the remaining nodes of the non-empty list
        curr.next = list1 if list1 else list2

        return dummy.next
```

43. [LeetCode 23: Merge k Sorted Lists](#) [Hard]

🔑 Key Points: Divide and Conquer / Min-Heap

💡 Intuition & Breaking Points:

Intuition & Pitfalls: We can reuse the logic of merging two sorted lists. Merging lists sequentially (e.g., merging list 1 with list 2, then the result with list 3, and so on) has an inefficient time complexity of $O(N * k)$, where N is the total number of nodes and k is the number of lists.

Mathematical Derivation: Divide and Conquer (similar to Merge Sort): Instead of merging sequentially, we merge lists in pairs: 1. Group the k lists into pairs and merge each pair. This reduces the number of lists from k to $k/2$. 2. Repeat this process recursively/iteratively: $k/2 \rightarrow k/4 \rightarrow k/8 \dots$ until only 1 merged list remains. In this divide and conquer approach, the total merge height is $\log k$ levels. At each level, every node is merged once, taking $O(N)$ time. Thus, the overall time complexity is optimized to $O(N \log k)$. This iterative method has $O(1)$ auxiliary space complexity, which is superior to min-heap solutions that require extra heap memory.

```
from typing import List, Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        # Time Complexity:  $O(N \log k)$  -  $N$  is the total number of nodes,  $k$  is the number of
        lists

        # Space Complexity:  $O(1)$  - Iterative divide and conquer merges in-place
        if not lists:
            return None

        # Helper function to merge two sorted lists
        def merge2Lists(l1, l2):
            dummy = ListNode(0)
            curr = dummy
            while l1 and l2:
                if l1.val <= l2.val:
```

```

        curr.next = l1
        l1 = l1.next
    else:
        curr.next = l2
        l2 = l2.next
    curr = curr.next
    curr.next = l1 if l1 else l2
    return dummy.next

# Pairwise iterative divide and conquer merge
interval = 1
n = len(lists)
while interval < n:
    for i in range(0, n - interval, interval * 2):
        lists[i] = merge2Lists(lists[i], lists[i + interval])
    interval *= 2

return lists[0]

```

44. [LeetCode 19: Remove Nth Node From End of List](#) [Medium]

 **Key Points:** Two Pointers - Fast & Slow Pointers Gap Control

Intuition & Breaking Points:

Intuition & Pitfalls: A naive approach is to perform a first pass to count the list length L , and a second pass to traverse to the $(L-N)$ -th node to remove it. This requires scanning the list twice. How can we achieve this in a single pass?

Mathematical Derivation: Two Pointers in a Single Pass: To remove the N -th node from the end, we need to locate its **immediate predecessor**. We can maintain a gap of N nodes between two pointers: 1. Initialize a `dummy` node pointing to `head` (to handle cases where the head itself is removed). 2. Place `fast` and `slow` pointers at the `dummy` node. 3. Advance `fast` by $N + 1$ steps. This establishes a gap of $N + 1$ nodes between `fast` and `slow`. 4. Advance both pointers at the same speed until `fast` reaches null (the end of the list). 5. At this point, `slow` will be

pointing exactly to the predecessor of the target node. We remove the target node by executing `slow.next = slow.next.next`. 6. Return `dummy.next`.

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        # Time Complexity: O(N) - Single pass traversal
        # Space Complexity: O(1)

        dummy = ListNode(0, head) # Dummy node to handle deletion of head node
        fast = dummy
        slow = dummy

        # Advance fast pointer by n + 1 steps to create a gap of n nodes
        for _ in range(n + 1):
            fast = fast.next

        # Move both pointers simultaneously until fast reaches the end
        while fast:
            fast = fast.next
            slow = slow.next

        # slow now points to the node preceding the target. Delete the target node.
        slow.next = slow.next.next

        return dummy.next
```

45. [LeetCode 143: Reorder List](#) [Medium]



Key Points: Find Midpoint (Fast & Slow Pointers) + Reverse List + Interleave Merge

Intuition & Breaking Points:

Intuition & Pitfalls: Reorder the list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$ to $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$. Using an auxiliary array to store nodes makes this trivial but requires $O(N)$ extra space. We need an $O(1)$ space solution.

Mathematical Derivation: Three-Step Splicing: Notice that the reordered list is formed by taking the second half of the list, reversing it, and interleaving it with the first half. 1. **Find midpoint**: Use fast/slow pointers. When the fast pointer reaches the end, the slow pointer points to the midpoint. Split the list into `first_half` and `second_half`. 2. **Reverse second half**: Reverse the `second_half` in-place. 3. **Interleave merge**: Interleave nodes from `first_half` and the reversed `second_half` one by one by adjusting their pointers. This requires no extra memory allocation.

```
from typing import Optional

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        # Do not return anything, modify head in-place instead.
        # Time Complexity: O(N) - Finding midpoint, reversing, and merging take O(N)
        combined

        # Space Complexity: O(1) - Pointers modified in-place
        if not head or not head.next:
            return

        # 1. Find midpoint using fast/slow pointers
        slow, fast = head, head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # slow points to the midpoint. Split the list.
        second_half = slow.next
        slow.next = None # Disconnect first half from second half
```

```
# 2. Reverse the second half of the list in-place
prev = None
curr = second_half
while curr:
    nxt = curr.next
    curr.next = prev
    prev = curr
    curr = nxt
second_half = prev # Head of the reversed second half

# 3. Interleave the first half and reversed second half
first_half = head
while first_half and second_half:
    # Store successors
    tmp1 = first_half.next
    tmp2 = second_half.next

    # Interleave pointers
    first_half.next = second_half
    second_half.next = tmp1

    # Advance pointers
    first_half = tmp1
    second_half = tmp2
```


Matrices

46. [LeetCode 73: Set Matrix Zeroes](#) [Medium]

 **Key Points:** Matrix In-Place Marking - $O(1)$ Space Optimization

Intuition & Breaking Points:

Intuition & Pitfalls: We need to set rows and columns to 0 if a cell contains 0. Modifying elements directly during the scan will corrupt state detection for downstream cells, causing the whole grid to turn to 0. Creating external row/column arrays requires $O(M + N)$ space. How can we optimize this to $O(1)$?

Mathematical Derivation: We can use the **first row and first column** of the matrix itself as the state markers: 1. Check if the first row and column initially contain any 0s, saving this state in two boolean flags `first_row_zero` and `first_col_zero`. 2. Iterate through the rest of the matrix (excluding the first row and column). If `matrix[r][c] == 0`, set its projection markers `matrix[r][0] = 0` and `matrix[0][c] = 0`. 3. Iterate through the sub-grid again. If row marker `matrix[r][0] == 0` or column marker `matrix[0][c] == 0` is set, change `matrix[r][c]` to 0. 4. Finally, use the boolean flags to zero out the first row and column if necessary. This achieves $O(1)$ auxiliary space.

```
from typing import List

class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        # Do not return anything, modify matrix in-place instead.
        # Time Complexity:  $O(M * N)$  - Two passes over the matrix
        # Space Complexity:  $O(1)$  - Mark state stored in first row and first column
        if not matrix:
            return
```

```
m, n = len(matrix), len(matrix[0])
first_row_zero = False
first_col_zero = False

# 1. Determine if the first row or first column has any zero initially
for c in range(n):
    if matrix[0][c] == 0:
        first_row_zero = True
        break
for r in range(m):
    if matrix[r][0] == 0:
        first_col_zero = True
        break

# 2. Iterate rest of the cells and store the zero states in the first row and column
for r in range(1, m):
    for c in range(1, n):
        if matrix[r][c] == 0:
            matrix[r][0] = 0
            matrix[0][c] = 0

# 3. Use the marks in the first row/col to zero out cells in the sub-matrix
for r in range(1, m):
    for c in range(1, n):
        if matrix[r][0] == 0 or matrix[0][c] == 0:
            matrix[r][c] = 0

# 4. Separately zero out the first row and first column if needed
if first_row_zero:
    for c in range(n):
        matrix[0][c] = 0
if first_col_zero:
    for r in range(m):
        matrix[r][0] = 0
```

47. [LeetCode 54: Spiral Matrix](#) [Medium]

🔑 Key Points: Matrix Boundary Simulation

💡 Intuition & Breaking Points:

Intuition & Pitfalls: We need to return all elements of the matrix in spiral (clockwise) order. This is a simulation task. We can maintain four boundaries (top, bottom, left, right) and shrink them as we complete traversal directions.

Mathematical Derivation: 1. Initialize the boundaries: `top = 0`, `bottom = m - 1`, `left = 0`, `right = n - 1`. 2. While `left <= right` and `top <= bottom`: - **Go right**: Traverse the top row from left to right, then increment `top`. - **Go down**: Traverse the right column from top to bottom, then decrement `right`. - **Go left**: (Verify `top <= bottom` first to prevent duplicate processing of a single row) Traverse the bottom row from right to left, then decrement `bottom`. - **Go up**: (Verify `left <= right` first to prevent duplicate processing of a single column) Traverse the left column from bottom to top, then increment `left`. 3. The simulation naturally terminates when boundaries overlap.

```
from typing import List

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        # Time Complexity: O(M * N) - Each cell is visited exactly once
        # Space Complexity: O(1) - Traversal pointer state uses constant space
        if not matrix:
            return []

        m, n = len(matrix), len(matrix[0])
        res = []

        # Define boundaries
        top, bottom = 0, m - 1
        left, right = 0, n - 1

        while left <= right and top <= bottom:
            # 1. Traverse from left to right along top boundary
            for col in range(left, right + 1):
                res.append(matrix[top][col])
            top += 1
```

```

# 2. Traverse from top to bottom along right boundary
for row in range(top, bottom + 1):
    res.append(matrix[row][right])
    right -= 1

if top <= bottom:
    # 3. Traverse from right to left along bottom boundary
    for col in range(right, left - 1, -1):
        res.append(matrix[bottom][col])
        bottom -= 1

if left <= right:
    # 4. Traverse from bottom to top along left boundary
    for row in range(bottom, top - 1, -1):
        res.append(matrix[row][left])
        left += 1

return res

```

48. [LeetCode 48: Rotate Image](#) [Medium]

 **Key Points:** In-Place Matrix Transpose + Row Reverse

Intuition & Breaking Points:

Intuition & Pitfalls: To rotate a matrix clockwise by 90 degrees, we could map elements into a new matrix using the index relation `new_matrix[j][n-1-i] = matrix[i][j]`. However, interviews require **in-place** rotation with $O(1)$ space constraint.

Mathematical Derivation: Using matrix transpose and line reflection: A 90-degree clockwise rotation can be decomposed into two simple geometric steps: 1. **Transpose the matrix**: Swap `matrix[i][j]` with `matrix[j][i]` for `j > i`. This turns rows into columns (in reversed order). 2. **Reverse each row**: Mirror-flip each row

horizontally (`matrix[i].reverse()`). Both steps are performed in-place using two-pointer swaps, requiring $O(1)$ auxiliary space.

```
from typing import List

class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        # Do not return anything, modify matrix in-place instead.
        # Time Complexity:  $O(N^2)$  - N is matrix dimension. Transpose and reverse take  $O(N^2)$ 
        # combined
        # Space Complexity:  $O(1)$  - Pointers modified in-place
        n = len(matrix)

        # 1. Transpose the matrix: Swap matrix[i][j] with matrix[j][i]
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

        # 2. Reverse each row horizontally
        for i in range(n):
            matrix[i].reverse()
```

49. [LeetCode 79: Word Search](#) [Medium]

 **Key Points:** Backtracking / DFS

Intuition & Breaking Points:

Intuition & Pitfalls: We need to find if a word path exists in a grid. Since we cannot reuse the same cell twice in a single path, this is a path search in an undirected graph. We scan for matching first letters, then initiate DFS.

Mathematical Derivation: 1. Iterate through each cell `(r, c)`. If `grid[r][c] == word[0]`, initiate DFS `dfs(r, c, 0)` where the index `0` represents the matching character index in the word. 2. DFS logic `dfs(r, c, index)`: - **Base case**: If `index ==`

len(word)`, the word is fully matched. Return True. - **Pruning**: If `(r, c)` is out of bounds, already visited, or `grid[r][c] != word[index]`, return False. - **Mark visited**: Temporarily mark the cell as visited by overwriting it with a special character (e.g. `'#'`). This avoids using a visited set. - **Four-directional search**: Recursively call DFS in four directions for `index + 1`. If any direction returns True, path exists. - **Backtrack**: Restore the original character to the cell (unmarking it) before returning.

```
from typing import List

class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        # Time Complexity: O(M * N * 3^L) - M, N are dimensions, L is word length. At each
        # cell, we explore 3 branches (excluding the parent path)
        # Space Complexity: O(L) - Maximum recursion stack depth equals word length L
        m, n = len(board), len(board[0])

        def dfs(r, c, index):
            # All letters matched successfully
            if index == len(word):
                return True

            # Boundary check and character match check
            if (r < 0 or r >= m or c < 0 or c >= n or
                board[r][c] != word[index]):
                return False

            # Temporarily mark current cell as visited
            temp = board[r][c]
            board[r][c] = '#'

            # Search adjacent cells recursively
            found = (dfs(r + 1, c, index + 1) or
                    dfs(r - 1, c, index + 1) or
                    dfs(r, c + 1, index + 1) or
                    dfs(r, c - 1, index + 1))

            # Restore original character (Backtracking)
            board[r][c] = temp
```

```
        return found

    for r in range(m):
        for c in range(n):
            # Trigger search from cells matching the first letter
            if board[r][c] == word[0] and dfs(r, c, 0):
                return True

    return False
```

Strings

50. [LeetCode 3: Longest Substring Without Repeating Characters](#)

[Medium]

 **Key Points:** Sliding Window / Hash Set

Intuition & Breaking Points:

Intuition & Pitfalls: We need to find the longest contiguous substring in which all characters are unique. A brute-force check of all substrings takes $O(N^2)$ time. To optimize this to $O(N)$, we can use a two-pointer sliding window.

Mathematical Derivation: Sliding Window Mechanism: We maintain a window defined by a `left` pointer and a `right` pointer, along with a hash set `char\_set` storing the unique characters inside this window. 1. The `right` pointer moves from left to right, expanding the window and adding new characters. 2. **Collision contraction**: If `s[right]` is already present in `char\_set`, we have a duplicate. We must shrink the window from the left by removing `s[left]` from `char\_set` and incrementing `left` until the duplicate character is removed from the window. 3. **Window validity**: Once the conflict is resolved, we add `s[right]` to `char\_set` and compute the current window size `right - left + 1`, updating the global maximum length `max\_len`. Each character is added and removed from the set at most once, resulting in an $O(N)$ time complexity.

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # Time Complexity: O(N) - Left and right pointers scan the string at most once
        # Space Complexity: O(min(M, N)) - M is the size of the character set, N is string
        length
        char_set = set()
        left = 0
```



```

max_len = 0

for right in range(len(s)):
    # Shrink window from the left if duplicate character is found
    while s[right] in char_set:
        char_set.remove(s[left])
        left += 1
    # Add the new unique character to the window
    char_set.add(s[right])
    # Update the maximum substring length seen so far
    max_len = max(max_len, right - left + 1)

return max_len

```

51. [LeetCode 424: Longest Repeating Character Replacement](#) [Medium]

 **Key Points:** Sliding Window - Dynamic Contraction & Max Freq Optimization

Intuition & Breaking Points:

Intuition & Pitfalls: We want to find the longest substring that can be converted to all identical characters by replacing at most k characters. If a window has length L and the highest frequency of any single character in it is max_freq , the number of operations needed is $L - \text{max_freq}$. The window is valid if $L - \text{max_freq} \leq k$.

Mathematical Derivation: Using an optimized Sliding Window: 1. Initialize $\text{left} = 0$, a dictionary count to store character frequencies in the window, and a variable $\text{max_freq} = 0$ to track the maximum frequency of any character seen in *any* window so far. 2. Iterate right across the string: - Increment $\text{count}[\text{s}[\text{right}]]$. - Update $\text{max_freq} = \max(\text{max_freq}, \text{count}[\text{s}[\text{right}]])$. - **Contraction check**: If the current window length minus max_freq exceeds k (i.e. $(\text{right} - \text{left} + 1) - \text{max_freq} > k$), the window is invalid. We shrink the window by decrementing $\text{count}[\text{s}[\text{left}]]$ and advancing left . - Why do we not decrement max_freq when shrinking the window? Because we are searching for a maximum length window. A smaller

`max\_freq` will only result in smaller windows, which cannot beat our historical maximum. Thus, `max\_freq` only needs to be updated when it increases.

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        # Time Complexity: O(N) - Single pass with left and right pointers
        # Space Complexity: O(1) - Dictionary size is capped at 26 uppercase English letters
        count = {}
        max_freq = 0
        left = 0
        max_len = 0

        for right in range(len(s)):
            count[s[right]] = count.get(s[right], 0) + 1
            # Update max frequency found in any window so far
            max_freq = max(max_freq, count[s[right]])

            # If replacement operations needed exceed k, shrink the window from the left
            while (right - left + 1) - max_freq > k:
                count[s[left]] -= 1
                left += 1

            max_len = max(max_len, right - left + 1)

        return max_len
```

52. [LeetCode 76: Minimum Window Substring](#) [Hard]

 **Key Points:** Sliding Window / Double Hash Maps / Optimal Shrinking

Intuition & Breaking Points:

Intuition & Pitfalls: To find the minimum window in `s` containing all characters of `t`, we can use a sliding window. We expand `right` until the window contains all

characters in `t` (forming a valid window), then we shrink `left` to discard unnecessary characters, optimizing for the smallest length.

Mathematical Derivation: 1. **Initialization**: Create a hash map `need` to store character frequencies in `t`, and `required` representing the number of unique characters in `t` that must be matched. Use a hash map `window` for characters in the current window, and `have` representing how many unique characters have met their target frequency. 2. **Expansion**: Move `right` to include `s[right]` in `window`. If `s[right]` is in `need` and its frequency in `window` matches its frequency in `need`, increment `have`. 3. **Contraction**: While `have == required` (window is valid), we attempt to shrink the window from the left: - Record the minimum window starting position and length. - Remove `s[left]` from `window`. - If the count of `s[left]` falls below its target in `need`, decrement `have`. - Increment `left` to shift the window start.

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        # Time Complexity: O(M + N) - M and N are lengths of s and t. Each char is visited
        # at most twice
        # Space Complexity: O(M + N) - Auxiliary space for need and window hash maps
        if not s or not t:
            return ""

        need = {}
        for c in t:
            need[c] = need.get(c, 0) + 1

        required = len(need)
        window = {}
        have = 0

        # Track minimum window length and its boundaries
        res = float('inf')
        res_indices = [-1, -1]

        left = 0
        for right in range(len(s)):
            char = s[right]
            window[char] = window.get(char, 0) + 1
```

```

# If the current character is required and its count matches the target
if char in need and window[char] == need[char]:
    have += 1

# Shrink window from the left as long as it remains valid
while have == required:
    if (right - left + 1) < res:
        res = right - left + 1
        res_indices = [left, right]

    # Pop left character out of the window
    left_char = s[left]
    window[left_char] -= 1
    if left_char in need and window[left_char] < need[left_char]:
        have -= 1
    left += 1

l, r = res_indices
return s[l:r+1] if res != float('inf') else ""

```

53. [LeetCode 242: Valid Anagram](#) [Easy]

 **Key Points:** Hash Map Frequencies / Character Counter

Intuition & Breaking Points:

Intuition & Pitfalls: An anagram is a word formed by rearranging the letters of another. This means both strings must contain the exact same character frequencies.

Mathematical Derivation: 1. Quick check: If lengths differ, they cannot be anagrams, return False. 2. Maintain a frequency map. Loop through both strings simultaneously: increment the frequency for `s[i]` and decrement for `t[i]`. 3. Traverse the frequency map values. If all frequencies are 0, return True; otherwise, return False.

```

class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        # Time Complexity: O(N) - Single pass through both strings of length N
        # Space Complexity: O(1) - Character set is capped at 26 lowercase English letters
        if len(s) != len(t):
            return False

        count = {}
        for i in range(len(s)):
            count[s[i]] = count.get(s[i], 0) + 1
            count[t[i]] = count.get(t[i], 0) - 1

        # If any character frequency is not balanced, return False
        for val in count.values():
            if val != 0:
                return False

        return True

```

54. [LeetCode 49: Group Anagrams](#) [Medium]

 **Key Points:** Hash Map - Sorted Key Normalization

Intuition & Breaking Points:

Intuition & Pitfalls: To group anagrams together, we need to map all strings that are anagrams of each other to a single, unique 'Signature' Key in a hash map.

Mathematical Derivation: Designing the Signature Key: 1. **Sorting**: Sort the characters of each string alphabetically. For example, 'eat', 'tea', and 'ate' all sort to 'aet'. We can use 'aet' as the hash map key. For a string of length L , sorting takes $O(L \log L)$ time. 2. **Counting**: Map each string to a 26-tuple of character counts: $(1, 0, 0, \dots, 1, \dots)$. Since tuples are immutable in Python, they are hashable and

can be used as keys. This takes $O(L)$ time. In practice, sorting is extremely simple and fast enough for interview settings.

```
from typing import List

class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        # Time Complexity:  $O(N * L \log L)$  - N is the number of strings, L is the maximum
        # length of a string
        # Space Complexity:  $O(N * L)$  - Space required to store the grouped strings in the
        # map
        anagrams_map = {}

        for s in strs:
            # Normalize key by sorting characters alphabetically
            sorted_s = "".join(sorted(s))
            if sorted_s not in anagrams_map:
                anagrams_map[sorted_s] = []
            anagrams_map[sorted_s].append(s)

        return list(anagrams_map.values())
```

55. [LeetCode 20: Valid Parentheses](#) [Easy]

 **Key Points:** Stack Data Structure - LIFO

Intuition & Breaking Points:

Intuition & Pitfalls: Parentheses matching follows a Last-In, First-Out (LIFO) pattern. The most recently opened bracket must be the first one closed. This matches the properties of a stack.

Mathematical Derivation: 1. Initialize an empty stack `stack` and a hash map `mapping` mapping closing brackets to their corresponding opening brackets: `mapping = {'(': ')', '{': '}', '[': ']'}`. 2. Traverse each character `char` in the string: - If

`char` is a closing bracket (in `mapping` keys): - Pop the top element from the stack (or use a placeholder like `'#'` if empty). If the popped element does not match the mapped opening bracket, return False. - If `char` is an opening bracket, push it onto the stack. 3. After checking the string, return True if the stack is empty (all brackets matched), otherwise False.

```
class Solution:
    def isValid(self, s: str) -> bool:
        # Time Complexity: O(N) - Single pass traversal. Push and pop operations are O(1)
        # Space Complexity: O(N) - In the worst case (e.g. all opening brackets), stack
        # stores all characters
        stack = []
        mapping = {"(": "(", ")": "(", "{": "{", "}": "}"

        for char in s:
            # If it's a closing bracket, validate against the top of the stack
            if char in mapping:
                top_element = stack.pop() if stack else '#'
                if mapping[char] != top_element:
                    return False
            else:
                # If it's an opening bracket, push onto stack
                stack.append(char)

        # Return True only if all opening brackets have been matched and popped
        return not stack
```

56. [LeetCode 125: Valid Palindrome](#) [Easy]

 **Key Points:** Two Pointers - In-Place Evaluation

 **Intuition & Breaking Points:**

Intuition & Pitfalls: We can strip out all non-alphanumeric characters, convert the remaining string to lowercase, and check if it equals its reverse. However, this takes $O(N)$ extra space. Can we do this in-place in $O(1)$ space?

Mathematical Derivation: Using two pointers on the original string: 1. Initialize `left = 0` and `right = len(s) - 1`. 2. While `left < right`: - Skip non-alphanumeric characters on the left: `left += 1`. - Skip non-alphanumeric characters on the right: `right -= 1`. - Compare `s[left].lower()` with `s[right].lower()`. If they are not equal, return False. - Move both pointers inward: `left += 1`, `right -= 1`. 3. If pointers cross without mismatch, return True. This avoids any extra memory allocations.

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Time Complexity: O(N) - In the worst case, pointers scan the string once
        # Space Complexity: O(1) - Constant auxiliary space
        left, right = 0, len(s) - 1

        while left < right:
            # Advance left pointer if current character is non-alphanumeric
            while left < right and not s[left].isalnum():
                left += 1

            # Decrement right pointer if current character is non-alphanumeric
            while left < right and not s[right].isalnum():
                right -= 1

            # Case-insensitive comparison
            if s[left].lower() != s[right].lower():
                return False

            left += 1
            right -= 1

        return True
```

57. [LeetCode 5: Longest Palindromic Substring](#) [Medium]

🔑 Key Points: Expand Around Center - O(1) Space Optimization

🧠 Intuition & Breaking Points:

Intuition & Pitfalls: A brute-force check of all substrings for palindromes takes $O(N^3)$ time. Standard 2D DP takes $O(N^2)$ time and $O(N^2)$ space. We can optimize space to $O(1)$ using center expansion.

Mathematical Derivation: Expand Around Center: Palindromes are symmetric around their center. We can treat each index (and gaps between indices) as a palindrome center and expand outwards:

1. Loop `i` from `0` to `len(s) - 1`.
2. **Odd length palindromes**: Expand from center `(i, i)` (e.g. "aba").
3. **Even length palindromes**: Expand from center `(i, i + 1)` (e.g. "abba").
4. Expand while `left >= 0` and `right < len(s)` and `s[left] == s[right]`. Record the maximum length palindrome found. Since there are only `2N - 1` possible centers, the total time complexity is $O(N^2)$ and space is $O(1)$.

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        # Time Complexity:  $O(N^2)$  -  $2N - 1$  centers, each can expand up to  $O(N)$  steps
        # Space Complexity:  $O(1)$  - Only pointers and bounds are tracked
        if not s or len(s) < 1:
            return ""

        res = ""

        # Helper function to expand from center and return the palindromic substring
        def expand(left, right):
            while left >= 0 and right < len(s) and s[left] == s[right]:
                left -= 1
                right += 1
            return s[left + 1:right]

        for i in range(len(s)):
            # Odd length center expansion
            p1 = expand(i, i)
            # Even length center expansion
            p2 = expand(i, i + 1)
```

```

        # Maintain the longest palindrome found
        if len(p1) > len(res):
            res = p1
        if len(p2) > len(res):
            res = p2

    return res

```

58. [LeetCode 647: Palindromic Substrings](#) [Medium]

 **Key Points:** Expand Around Center - Palindrome Counter

Intuition & Breaking Points:

Intuition & Pitfalls: We need to find the total count of palindromic substrings. Since every palindrome expands around a center, we can count palindromes by initiating center expansions and incrementing a counter for each successful step.

Mathematical Derivation: Like the longest palindromic substring problem, we use $2N - 1$ centers: 1. Iterate i from 0 to $n - 1$. 2. **Odd length center**: Expand from (i, i) . Increment count each time we expand successfully. 3. **Even length center**: Expand from $(i, i + 1)$. Increment count each time we expand successfully. 4. Return the total count. This achieves $O(N^2)$ time and $O(1)$ space.

```

class Solution:
    def countSubstrings(self, s: str) -> int:
        # Time Complexity:  $O(N^2)$  -  $2N - 1$  centers, each can expand up to  $O(N)$  steps
        # Space Complexity:  $O(1)$ 

        count = 0
        n = len(s)

        def expand_and_count(left, right):
            nonlocal count
            while left >= 0 and right < n and s[left] == s[right]:
                count += 1
                left += 1
                right += 1

```

```

        left -= 1
        right += 1

    for i in range(n):
        # Check odd length palindromes centered at index i
        expand_and_count(i, i)

        # Check even length palindromes centered between i and i + 1
        expand_and_count(i, i + 1)

    return count

```

59. [LeetCode 271: Encode and Decode Strings](#) [Medium]

 **Key Points:** Serialization & Deserialization / Length-Prefixed Protocol

Intuition & Breaking Points:

Intuition & Pitfalls: We need to serialize a list of strings into a single string such that it can be decoded back. Using simple delimiters (like comma or semicolon) fails if the strings themselves contain those delimiters.

Mathematical Derivation: Using a **Length-Prefixed Protocol** avoids delimiter collision: 1. **Encode**: For each string `s`, we prepend its length `len(s)` followed by a non-digit delimiter (such as `#`), and then the string itself: `[length] + '#' + s`. For example, `["hello", "world"]` encodes to `"5#hello5#world"`. If a string is `"ab#c"`, it encodes to `"4#ab#c"` without ambiguity because we know exactly how many characters to read after the delimiter. 2. **Decode**: Maintain a pointer `i = 0` to scan the encoded string: - Find the first `#` starting from index `i`. - Read the substring before `#` and convert it to an integer, which is the `length` of the string. - Read the next `length` characters starting immediately after `#` to retrieve the original string. - Move the pointer to the end of the extracted string: `i = sharp_idx + 1 + length`.

```
from typing import List
```

```
class Codec:

    def encode(self, strs: List[str]) -> str:

        # Encodes a list of strings to a single string.

        # Time Complexity: O(N) - N is total length of all strings

        # Space Complexity: O(1) - Excluding output allocation

        res = []

        for s in strs:

            # Append length + '#' + string itself

            res.append(f"{len(s)}#{s}")

        return "".join(res)

    def decode(self, s: str) -> List[str]:

        # Decodes a single string to a list of strings.

        # Time Complexity: O(N) - Single pass to parse the string

        # Space Complexity: O(1)

        res = []

        i = 0

        while i < len(s):

            # Locate the length delimiter

            sharp_idx = s.find('#', i)

            length = int(s[i:sharp_idx])

            # Slice the string according to the parsed length prefix

            start = sharp_idx + 1

            end = start + length

            res.append(s[start:end])

            # Advance pointer past the processed string segment

            i = end

        return res
```

Trees

60. [LeetCode 104: Maximum Depth of Binary Tree](#) [Easy]

 **Key Points:** Binary Tree Recursion - Depth-First Search (DFS)

Intuition & Breaking Points:

Intuition & Pitfalls: The maximum depth of a binary tree is the maximum of the depths of its left and right subtrees, plus 1 for the root node itself.

Mathematical Derivation: We can express this relation recursively: 1. **Base case**: If `root` is null, the subtree is empty, so return height 0. 2. **Recursion**: Recursively calculate the maximum depth of the left subtree `left_depth` and the right subtree `right_depth`. 3. **Combine**: The result is `max(left_depth, right_depth) + 1`.

```
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        # Time Complexity: O(N) - Each node is visited exactly once
        # Space Complexity: O(H) - H is the height of the tree, representing the recursion
        # stack depth
        if not root:
```

```
        return 0

    return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

61. [LeetCode 100: Same Tree](#) [Easy]

 **Key Points:** Binary Tree - Synchronous Recursion

Intuition & Breaking Points:

Intuition & Pitfalls: For two trees to be identical, their root values must be equal, and their left subtrees must be identical, and their right subtrees must be identical. We can compare them using synchronous recursion.

Mathematical Derivation: Compare two nodes `p` and `q` simultaneously: 1. If both `p` and `q` are null, we reached the leaf ends simultaneously with matching structures, return True. 2. If only one is null, or their values differ (`p.val != q.val`), they are not identical, return False. 3. Otherwise, recursively verify that the left subtrees match `self.isSameTree(p.left, q.left)` and the right subtrees match `self.isSameTree(p.right, q.right)`. Both must be True.

```
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        # Time Complexity: O(N) - N is the smaller number of nodes in p and q
        # Space Complexity: O(H) - Recursion stack space
        if not p and not q:
            return True
        if not p or not q or p.val != q.val:
```

```

        return False

    # Check left and right subtrees recursively
    return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)

```

62. [LeetCode 226: Invert Binary Tree](#) [Easy]

 **Key Points:** Binary Tree - In-Place Node Swapping

Intuition & Breaking Points:

Intuition & Pitfalls: Inverting a binary tree (mirroring it) means swapping the left child and right child of every node in the tree recursively.

Mathematical Derivation: 1. **Base case:** If the current node `root` is null, return `None`. 2. **Swap children:** Swap the left and right pointers of the current node using a parallel assignment: `root.left, root.right = root.right, root.left`. 3. **Recursive step:** Recursively invert the left subtree `self.invertTree(root.left)` and the right subtree `self.invertTree(root.right)`. 4. Return the mutated `root` node.

```

from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        # Time Complexity: O(N) - Visit every node once
        # Space Complexity: O(H) - Recursion stack space
        if not root:
            return None

```

```
# Swap the left and right child pointers
root.left, root.right = root.right, root.left

# Recursively invert the children
self.invertTree(root.left)
self.invertTree(root.right)

return root
```

63. [LeetCode 124: Binary Tree Maximum Path Sum](#) [Hard]

 **Key Points:** Post-Order DFS / Path Contribution Merging

Intuition & Breaking Points:

Intuition & Pitfalls: A path in a binary tree can 'bend' at a node, meaning it can travel from the left subtree, through the node, and into the right subtree. For any node, the maximum sum of a path bending at this node is `node.val + left_gain + right_gain`. However, a path with a bend cannot be extended upward to a parent node because binary tree paths cannot have branches.

Mathematical Derivation: Post-Order DFS Mechanism: 1. Maintain a global variable `max_sum` to store the maximum path sum found, initialized to negative infinity. 2. Define a DFS helper that returns the **maximum single-path contribution** the subtree can provide to its parent (i.e. we must choose only *one* branch to extend upward):

- If the node is null, return 0.
- Calculate the max gain from the left child: `left = max(0, dfs(node.left))`. If a child's gain is negative, we discard it (greedy choice).
- Calculate the max gain from the right child: `right = max(0, dfs(node.right))`.
- **Calculate the bent path:** The max path sum bending at the current node is `node.val + left + right`. Update `max_sum` with this value.
- **Return single-path gain:** Return `node.val + max(left, right)` to the parent node since the path cannot branch.


```
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        # Time Complexity: O(N) - Each node is visited exactly once during DFS
        # Space Complexity: O(H) - Auxiliary recursion stack space
        max_sum = float('-inf')

        def dfs(node):
            nonlocal max_sum
            if not node:
                return 0

            # Compute maximum single-path gain from left and right children (ignore negative gains)
            left_gain = max(dfs(node.left), 0)
            right_gain = max(dfs(node.right), 0)

            # Update the global maximum path sum considering a path that splits at the current node
            current_path_sum = node.val + left_gain + right_gain
            max_sum = max(max_sum, current_path_sum)

            # Return the maximum single-path gain (cannot take both left and right to the parent)
            return node.val + max(left_gain, right_gain)

        dfs(root)
        return max_sum
```

64. [LeetCode 102: Binary Tree Level Order Traversal](#) [Medium]

 **Key Points:** Breadth-First Search (BFS) / Queue Level Splitting

Intuition & Breaking Points:

Intuition & Pitfalls: To output nodes level by level, we use a Breadth-First Search (BFS) facilitated by a queue. When processing a level, we must determine the size of the queue at the start of the level. This allows us to process exactly the nodes belonging to the current level before moving to the next.

Mathematical Derivation: 1. If `root` is null, return an empty list `[]`. 2. Initialize a double-ended queue `queue` containing the `root`. 3. While the queue is not empty: - Get the number of nodes in the current level: `level\_size = len(queue)`. - Iterate `level\_size` times. Dequeue a node, append its value to the current level's list `level\_nodes`, and enqueue its non-null left and right children. - Append `level\_nodes` to the global result list.

```
from typing import List, Optional
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        # Time Complexity: O(N) - Visit every node once
        # Space Complexity: O(N) - Queue holds at most N/2 nodes at the bottom level
        if not root:
            return []

        res = []
        queue = deque([root])
```

```

while queue:
    level_size = len(queue) # Number of nodes at the current level
    current_level = []

    for _ in range(level_size):
        curr = queue.popleft()
        current_level.append(curr.val)
        # Enqueue children for the next level
        if curr.left:
            queue.append(curr.left)
        if curr.right:
            queue.append(curr.right)
    res.append(current_level)

return res

```

65. [LeetCode 297: Serialize and Deserialize Binary Tree](#) [Hard]

 **Key Points:** Pre-Order DFS / Tree Splicing / Serialization Protocol

Intuition & Breaking Points:

Intuition & Pitfalls: To serialize a binary tree into a string and deserialize it back, the serialized string must store the structure (including null nodes) explicitly. A traversal string without null markers cannot uniquely reconstruct a binary tree.

Mathematical Derivation: Pre-Order DFS (Root -> Left -> Right) Protocol: 1.

****Serialize**:** - Run pre-order DFS on the tree. If a node is null, append a placeholder like `'#'` or `'#'` to the result. - If the node is not null, append its value followed by a delimiter (like `','`). - Example: A tree is serialized to `1,2,##,3,4,##,5,##`. 2.

****Deserialize**:** - Split the serialized string by `','` and convert it into a queue `queue`. - Define a recursive helper `dfs(queue)` that dequeues the first element: - If it is `'#'`, return `None`. - Otherwise, instantiate `node = TreeNode(int(val))`. - Recursively construct the left child: `node.left = dfs()`. - Recursively construct the right child: `node.right = dfs()`. - Return `node`.

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Codec:
    def serialize(self, root):
        # Encodes a tree to a single string.
        # Time Complexity: O(N)
        # Space Complexity: O(N)
        res = []
        def dfs(node):
            if not node:
                res.append("#")
                return
            res.append(str(node.val))
            dfs(node.left)
            dfs(node.right)
        dfs(root)
        return ",".join(res)

    def deserialize(self, data):
        # Decodes your encoded data to tree.
        # Time Complexity: O(N)
        # Space Complexity: O(N)
        vals = deque(data.split(','))

        def dfs():
            val = vals.popleft()
            if val == '#':
                return None
            node = TreeNode(int(val))
            # Construct subtrees in pre-order sequence
            node.left = dfs()
            node.right = dfs()
            return node
```

```
return dfs()
```

66. [LeetCode 572: Subtree of Another Tree](#) [Easy]

 **Key Points:** Binary Tree Recursion - Substructure Matching

Intuition & Breaking Points:

Intuition & Pitfalls: To check if `subRoot` is a subtree of `root`, three cases are possible: 1. `root` and `subRoot` are identical trees. 2. `subRoot` is a subtree of `root.left`. 3. `subRoot` is a subtree of `root.right`.

Mathematical Derivation: 1. Helper `isSameTree(p, q)`: Returns True if two trees have identical structures and values. 2. Main function `isSubtree(root, subRoot)`: - If `root` is null, we reached the end of the search path without finding a match. Return False (since `subRoot` is non-empty). - Check `isSameTree(root, subRoot)`. If True, return True. - Otherwise, recursively search in the child nodes: `self.isSubtree(root.left, subRoot)` or `self.isSubtree(root.right, subRoot)`.

```
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode]) -> bool:
        # Time Complexity: O(M * N) - M, N are node counts. Worst case checks isSame for all
        # root nodes
        # Space Complexity: O(H) - H is height of the root tree (recursion stack depth)
        if not root:
            return False
```

```

def isSame(p, q):
    if not p and not q:
        return True
    if not p or not q or p.val != q.val:
        return False
    return isSame(p.left, q.left) and isSame(p.right, q.right)

# Check if identical at current root, otherwise recurse left and right
if isSame(root, subRoot):
    return True
return self.isSubtree(root.left, subRoot) or self.isSubtree(root.right, subRoot)

```

67. [LeetCode 105: Construct Binary Tree from Preorder and Inorder Traversal](#) [Medium]

 **Key Points:** Pre-Order & In-Order Properties / Divide and Conquer / Index Mapping

Intuition & Breaking Points:

Intuition & Pitfalls: In a pre-order traversal, the first element is always the **root node** of the current tree/subtree. In an in-order traversal, the root node partitions the elements into the left subtree (all elements to the left of the root) and the right subtree (all elements to the right). We can use this property to recursively rebuild the tree.

Mathematical Derivation: 1. **Root Extraction**: The first element in preorder is the root value: `root_val = preorder[pre_start]`. Instantiate `root = TreeNode(root_val)`. 2. **Partitioning**: Locate the index of `root_val` in inorder, say `mid`. The elements to the left of `mid` represent the left subtree, and elements to the right represent the right subtree. 3. **Subproblem division**: The left subtree has size `left_size = mid - in_start`. Thus, the left subtree's preorder values are located at indices `[pre_start + 1 ... pre_start + left_size]`, and the right subtree's preorder values are at indices `[pre_start + left_size + 1 ... pre_end]`. 4. **Optimization**: Slicing arrays in recursion takes $O(N^2)$ time. We optimize this to $O(N)$ by storing inorder values in a hash map

(`inorder_map = {val: idx}`) for $O(1)$ index lookup and passing boundary pointers in recursion.

```
from typing import List, Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        # Time Complexity:  $O(N)$  -  $N$  is the number of nodes. Hash map index lookup is  $O(1)$ 
        # Space Complexity:  $O(N)$  - Hash map storage and recursion call stack

        # Cache inorder indices to avoid  $O(N)$  searches during recursion
        inorder_map = {val: i for i, val in enumerate(inorder)}

        def build(pre_start, pre_end, in_start, in_end):
            if pre_start > pre_end:
                return None

            root_val = preorder[pre_start]
            root = TreeNode(root_val)

            # Locate root in inorder list
            mid = inorder_map[root_val]
            left_size = mid - in_start # Number of nodes in left subtree

            # Recursively build left and right subtrees
            root.left = build(pre_start + 1, pre_start + left_size, in_start, mid - 1)
            root.right = build(pre_start + left_size + 1, pre_end, mid + 1, in_end)

            return root

        return build(0, len(preorder) - 1, 0, len(inorder) - 1)
```

68. [LeetCode 98: Validate Binary Search Tree](#) [Medium]

 **Key Points:** BST Definition - Interval Bounds DFS Propagation

Intuition & Breaking Points:

Intuition & Pitfalls: A common mistake is validating only that a node is greater than its left child and smaller than its right child. This is incorrect. In a Binary Search Tree (BST), **all nodes in the left subtree must be less than the parent node, and all nodes in the right subtree must be greater than the parent node**.

Mathematical Derivation: To enforce this definition globally, we propagate valid open intervals `(lower, upper)` down during DFS:

1. Initialize `validate(root, -inf, inf)`.
2. In each step:
 - If the node is null, return True.
 - Verify if the node's value falls inside the open interval: `lower < node.val < upper`. If not, return False.
 - **Update bounds for children**:
 - When validating the left child, its value must be less than the current node's value. The upper bound shifts to `node.val`: `validate(node.left, lower, node.val)`.
 - When validating the right child, its value must be greater than the current node's value. The lower bound shifts to `node.val`: `validate(node.right, node.val, upper)`.
 - Return True if both children are valid BSTs.

```
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        # Time Complexity: O(N) - Visit every node once
        # Space Complexity: O(H) - Recursion stack space
        def validate(node, lower=float('-inf'), upper=float('inf')):
            if not node:
                return True
            if not (lower < node.val < upper):
                return False
            if not (validate(node.left, lower, node.val) and
                    validate(node.right, node.val, upper)):
                return False
            return True
```



```

# Node value must be strictly within the allowed range
if not (lower < node.val < upper):
    return False

# Left child values must be in (lower, node.val); right child values in
(node.val, upper)
return (validate(node.left, lower, node.val) and
        validate(node.right, node.val, upper))

return validate(root)

```

69. [LeetCode 230: Kth Smallest Element in a BST](#) [Medium]

 **Key Points:** BST In-Order Traversal Properties / Iterative Stack with Early Termination

Intuition & Breaking Points:

Intuition & Pitfalls: An in-order traversal (Left -> Root -> Right) of a Binary Search Tree (BST) visits nodes in strictly ascending order. Thus, finding the k-th smallest element is equivalent to finding the k-th node visited during an in-order traversal.

Mathematical Derivation: To optimize performance, we use an **iterative in-order traversal** utilizing an explicit stack. This allows us to stop the traversal immediately once the k-th element is reached:

1. Initialize an empty stack and set `curr = root`.
2. In a loop while `curr` is not null or the stack is not empty:
 - **Go left**: Push the current node and all its left descendants onto the stack: `stack.append(curr); curr = curr.left`. This ensures we always process the smallest element next.
 - **Pop and process**: Pop from the stack. This is the next smallest element in in-order. Decrement `k` by 1.
 - **Check target**: If `k == 0`, we have found the k-th smallest element. Return its value immediately.
 - **Go right**: Set `curr = popped_node.right` to process its right subtree.

```
from typing import Optional
```

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        # Time Complexity: O(H + k) - H is the height of the tree. We traverse down to the
        # leftmost leaf, then perform k pop operations
        # Space Complexity: O(H) - Stack stores at most H nodes
        stack = []
        curr = root

        while curr or stack:
            # 1. Travel to the leftmost node, pushing all nodes along the path to the stack
            while curr:
                stack.append(curr)
                curr = curr.left

            # 2. Process the top of the stack (smallest unvisited node)
            curr = stack.pop()
            k -= 1

            # 3. If we've popped k elements, we have found the target
            if k == 0:
                return curr.val

            # 4. Move to the right child
            curr = curr.right

        return -1

```

70. [LeetCode 235: Lowest Common Ancestor of a Binary Search Tree](#)

[Medium]

 **Key Points:** BST Partial Order / Binary Search-like Node Splitting

💡 Intuition & Breaking Points:

Intuition & Pitfalls: Finding the Lowest Common Ancestor (LCA) in a general binary tree takes $O(N)$ time and $O(H)$ space. However, because this is a Binary Search Tree (BST), we can use the value ordering property to find the LCA in $O(H)$ time and $O(1)$ space.

Mathematical Derivation: In a BST, a node's left child has a smaller value and its right child has a larger value. Traverse starting from the `root`:

1. If both `p.val` and `q.val` are less than the current node's value, the LCA must reside in the left subtree. Move left: `curr = curr.left`.
2. If both `p.val` and `q.val` are greater than the current node's value, the LCA must reside in the right subtree. Move right: `curr = curr.right`.
3. ****Split Point**:** If neither condition is met (one value is larger and the other is smaller, or one value equals the current node's value), the paths to `p` and `q` split here. The current node is the LCA. Return it.

```
class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        # Time Complexity: O(H) - We traverse down a single branch of the tree. H is O(log N) for balanced trees
        # Space Complexity: O(1) - Constant auxiliary space
        curr = root

        while curr:
            # If both targets are in the left subtree
            if p.val < curr.val and q.val < curr.val:
                curr = curr.left

            # If both targets are in the right subtree
            elif p.val > curr.val and q.val > curr.val:
                curr = curr.right

            else:
                # Split point found (or current is one of p or q)
```

```
        return curr

    return None
```

71. [LeetCode 208: Implement Trie \(Prefix Tree\)](#) [Medium]

 **Key Points:** Trie Structure / Nested Dictionaries

Intuition & Breaking Points:

Intuition & Pitfalls: A Trie (Prefix Tree) is a tree structure used for fast string retrieval and autocomplete queries. Each node in the Trie represents a character. A path from the root to a node spells out a word.

Mathematical Derivation: Trie Node Structure Design: We can implement a Trie node using a dictionary `children` mapping characters to child `TrieNode` objects, and a boolean flag `is\_word` indicating if the node marks the end of a complete word.

1. `insert(word)`: Iterate through the word's characters. If a character is not in the current node's `children`, insert a new `TrieNode`. Move the pointer to the child node. At the end of the word, set `is_word = True`.
2. `search(word)`: Traverse the Trie following the word's characters. If a character is missing, return False. At the end, return `is_word` (which must be True for the word to exist).
3. `startsWith(prefix)`: Traverse the Trie following the prefix. If the traversal succeeds, the prefix exists. Return True.

```
class TrieNode:
    def __init__(self):
        # Map of children nodes {char: TrieNode}
        self.children = {}
        # Boolean to check if node marks the end of a word
        self.is_word = False

class Trie:
    def __init__(self):
```

```
self.root = TrieNode()

def insert(self, word: str) -> None:
    # Time Complexity: O(L) - L is word length
    curr = self.root
    for char in word:
        if char not in curr.children:
            curr.children[char] = TrieNode()
        curr = curr.children[char]
    curr.is_word = True

def search(self, word: str) -> bool:
    # Time Complexity: O(L)
    curr = self.root
    for char in word:
        if char not in curr.children:
            return False
        curr = curr.children[char]
    return curr.is_word

def startsWith(self, prefix: str) -> bool:
    # Time Complexity: O(L)
    curr = self.root
    for char in prefix:
        if char not in curr.children:
            return False
        curr = curr.children[char]
    return True
```

72. [LeetCode 211: Add and Search Word](#) [Medium]

 **Key Points:** Trie + Wildcard DFS Backtracking

 **Intuition & Breaking Points:**

Intuition & Pitfalls: This problem adds wildcard searches to the standard Trie, where `.` can match any single character. While adding words is identical to insertion in a Trie, searching requires a DFS-based backtracking approach to handle the `.` character.

Mathematical Derivation: 1. Node structure: Standard Trie node (`children` dict, `is\_word` boolean). 2. `addWord(word)`: Standard Trie word insertion. 3. `search(word)`: Define a recursive helper `dfs(node, i)` that checks matching starting from character index `i` of `word` at Trie node `node`: - If `i == len(word)`, return `node.is\_word`. - Let `char = word[i]`: - **Wildcard dot**: If `char == '.'`, we must try all branches in `node.children.values()`. If any branch returns True for `dfs(child, i + 1)`, search is successful. Return True. If all fail, return False. - **Standard character**: If `char` exists in `node.children`, recursively check the child: `dfs(node.children[char], i + 1)`. Otherwise, return False.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_word = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        # Time Complexity: O(L) - L is word length
        curr = self.root
        for char in word:
            if char not in curr.children:
                curr.children[char] = TrieNode()
            curr = curr.children[char]
        curr.is_word = True

    def search(self, word: str) -> bool:
        # Time Complexity: O(M) worst case - M is total nodes in the Trie (wildcard search),
        # usually O(L)
        def dfs(node, i):
            if i == len(word):
                return node.is_word

            if word[i] == '.':
                for child in node.children.values():
                    if dfs(child, i + 1):
                        return True
            else:
                if word[i] in node.children:
                    if dfs(node.children[word[i]], i + 1):
                        return True
            return False

        return dfs(self.root, 0)
```

```

        char = word[i]
        if char == '.':
            # Wildcard search: try all possible child paths
            for child in node.children.values():
                if dfs(child, i + 1):
                    return True
            return False
        else:
            # Direct character match
            if char not in node.children:
                return False
            return dfs(node.children[char], i + 1)

    return dfs(self.root, 0)

```

73. [LeetCode 212: Word Search II](#) [Hard]

 **Key Points:** Trie + Grid DFS Backtracking + Node Deletion Pruning

Intuition & Breaking Points:

Intuition & Pitfalls: This is Word Search I with a list of words. Searching for each word individually takes $O(W * M * N * 3^L)$ time (where W is word count), which will time out. We must search for all words in a single grid scan.

Mathematical Derivation: Trie-Accelerated Grid DFS Backtracking: Instead of looping through words, we **store all target words in a Trie**. During the grid DFS traversal, we synchronize the grid position shifts with navigation in the Trie.

- Trie building:** Insert all words into the Trie. For convenience, store the actual word string at its leaf node: `node.word = word`.
- Grid DFS:** At `(r, c)`: - If `board[r][c] = char` is in the Trie node's children, move to `child_node = node.children[char]`. - If `child_node` has a `word` attribute, we found a match. Add it to results and **immediately clear `child_node.word = None` to prevent duplicate matches**. - Mark the grid cell as visited (`board[r][c] = '#'`), recursively call DFS in 4 directions, then restore the cell's character (backtrack).
- Trie Pruning Optimization:** If a node has no children (`not curr_node.children`), it is a dead end. We can delete it

from its parent's `children` map on backtracking. This pruning prevents exploring redundant grid paths and is crucial to passing strict constraints.

```
from typing import List

class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None # Stores the word if the node marks a word's end

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        # Time Complexity:  $O(M * N * 4 * 3^{(L-1)})$  - M, N are board dimensions, L is maximum word length
        # Space Complexity:  $O(W * L)$  - Space to store words in the Trie

        # 1. Build the Trie
        root = TrieNode()
        for word in words:
            curr = root
            for char in word:
                if char not in curr.children:
                    curr.children[char] = TrieNode()
                curr = curr.children[char]
            curr.word = word

        m, n = len(board), len(board[0])
        res = []

        # 2. Backtracking DFS
        def dfs(r, c, parent_node):
            char = board[r][c]
            curr_node = parent_node.children[char]

            # Match found, record word and clear to prevent duplicate matches
            if curr_node.word:
                res.append(curr_node.word)
                curr_node.word = None

            # Temporarily mark grid cell as visited
```



```
board[r][c] = '#'

# Explore 4 directions
for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
    nr, nc = r + dr, c + dc
    if 0 <= nr < m and 0 <= nc < n:
        next_char = board[nr][nc]
        if next_char in curr_node.children:
            dfs(nr, nc, curr_node)

# Restore grid cell (Backtrack)
board[r][c] = char

# Trie Pruning: Remove leaf nodes with no children
if not curr_node.children:
    parent_node.children.pop(char)

# 3. Scan the grid and trigger DFS from matching start characters
for r in range(m):
    for c in range(n):
        start_char = board[r][c]
        if start_char in root.children:
            dfs(r, c, root)

return res
```

Heaps

74. [LeetCode 347: Top K Frequent Elements](#) [Medium]

 **Key Points:** Hash Map Frequencies + Bucket Sort $O(N)$ Optimal / Min-Heap $O(N \log K)$

Intuition & Breaking Points:

Intuition & Pitfalls: We can count the frequency of each element using a hash map and sort the frequencies, taking $O(N \log N)$ time. Alternatively, we can maintain a min-heap of size K containing the highest frequency elements. Pushing new elements and adjusting takes $O(N \log K)$ time.

Mathematical Derivation: Using Bucket Sort for an optimal $O(N)$ solution: Since frequencies are bounded by $[0, N]$ (where N is the array length), we can use the concept of bucket sort: 1. Count element frequencies using a hash map. 2. Create an array of buckets of size $N + 1$, where `buckets[i]` stores all elements that have a frequency of exactly i . 3. Iterate over the frequency map, adding each element to its corresponding bucket list `buckets[freq]`. 4. **Reverse scan**: Scan the buckets from right to left (from highest frequency N to lowest 0), collecting elements until we have compiled k elements. This avoids sorting, taking $O(N)$ time and $O(N)$ space, which is optimal.

```
from typing import List

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # Time Complexity: O(N) - Frequency counting, bucket filling, and reverse scanning
        # take linear time
        # Space Complexity: O(N) - Auxiliary space for hash map and buckets array
```

```

# 1. Count occurrences of each number
count = {}
for num in nums:
    count[num] = count.get(num, 0) + 1

# 2. Create frequency buckets (index represents frequency)
n = len(nums)
buckets = [[] for _ in range(n + 1)]
for num, freq in count.items():
    buckets[freq].append(num)

# 3. Collect the top k frequent elements by scanning backwards from highest
frequency
res = []
for i in range(n, -1, -1):
    for num in buckets[i]:
        res.append(num)
        if len(res) == k:
            return res

return res

```

75. [LeetCode 295: Find Median from Data Stream](#) [Hard]

 **Key Points:** Dual Heaps Balance Design / Max-Heap & Min-Heap

Intuition & Breaking Points:

Intuition & Pitfalls: To find the median of a dynamic data stream, sorting on every insertion takes $O(N \log N)$ time. Using insertion sort to maintain a sorted array takes $O(N)$ insertion time. We need a faster solution that inserts and balances in $O(\log N)$ time.

Mathematical Derivation: Two Heaps (Max-Heap and Min-Heap) approach: We split the numbers into two equal-sized symmetric halves: - **Left half** (smaller values): Stored in a **Max-Heap** `small` (represented using negative values in Python

heapq). The root represents the maximum value in the left half. - **Right half** (larger values): Stored in a **Min-Heap** `large`. The root represents the minimum value in the right half. To insert a new element `num`: 1. Push `-num` into the Max-Heap `small`. 2. Balance the heap boundaries: Pop the maximum element from `small` and push it to the Min-Heap `large`. 3. Enforce the size constraint `len(small) >= len(large)` (meaning `small` can hold at most 1 more element than `large`). If `len(large) > len(small)`, pop the minimum element from `large` and push its negative to `small`. 4. **Get Median**: - If `len(small) > len(large)`, return the root of `small` (negated). - If `len(small) == len(large)`, return `(root of small + root of large) / 2.0`.

```
import heapq

class MedianFinder:
    def __init__(self):
        # max-heap (values negated to simulate max-heap with heapq)
        self.small = []
        # min-heap
        self.large = []

    def addNum(self, num: int) -> None:
        # Time Complexity: O(log N) - Heap push and pop operations
        # Push to max-heap first
        heapq.heappush(self.small, -num)

        # Ensure values in small (left half) are <= values in large (right half)
        val = -heapq.heappop(self.small)
        heapq.heappush(self.large, val)

        # Balance heap sizes to satisfy: len(small) >= len(large)
        if len(self.large) > len(self.small):
            val = heapq.heappop(self.large)
            heapq.heappush(self.small, -val)

    def findMedian(self) -> float:
        # Time Complexity: O(1) - Root operations are constant time
        if len(self.small) > len(self.large):
            # Odd count: median is the top of the max-heap (negated)
            return float(-self.small[0])
```

```
# Even count: median is the average of the two heap roots  
return (-self.small[0] + self.large[0]) / 2.0
```

